

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM**

**KHOA KỸ THUẬT MÁY TÍNH**



**BÁO CÁO ĐỒ ÁN MÔN HỌC**  
**ĐỀ TÀI: CACHE CONTROLLER**

**Môn học:** CE213.Q22 – THIẾT KẾ HỆ THỐNG SỐ VỚI HDL

**Giảng viên hướng dẫn:** ThS. Hồ Ngọc Diễm

**Thực hiện bởi nhóm, bao gồm:**

1. Trịnh Nguyễn Thanh Bình 23520162
2. Võ Hồ Thiên Đăng 23520239
3. Bùi Quốc Bảo 23520092

**Thời gian hoàn thành:** 18/05/2026

## MỤC LỤC

|                                                                                      |    |
|--------------------------------------------------------------------------------------|----|
| MỤC LỤC .....                                                                        | 2  |
| DANH SÁCH HÌNH, BẢNG .....                                                           | 5  |
| TÓM TẮT .....                                                                        | 6  |
| Chương I. TỔNG QUAN .....                                                            | 7  |
| 1. Tóm tắt đề tài .....                                                              | 7  |
| 2. Mục tiêu đề tài .....                                                             | 7  |
| 3. Phạm vi và giới hạn đề tài .....                                                  | 7  |
| Chương II. CƠ SỞ LÝ THUYẾT VÀ KIẾN TRÚC HỆ THỐNG .....                               | 9  |
| 1. Tổng quan về Bộ nhớ đệm (Cache Memory) .....                                      | 9  |
| 2. Phương pháp Ánh xạ Ánh xạ trực tiếp (Direct-Mapped Cache) .....                   | 9  |
| 3. Chính sách ghi Write-Back và Cấp phát Write-Allocate .....                        | 9  |
| 3.1. Chính sách ghi Write-Back .....                                                 | 9  |
| 3.2. Cấp phát Write-Allocate .....                                                   | 9  |
| 4. Công nghệ sử dụng .....                                                           | 9  |
| 5. Thuật toán điều khiển FSM .....                                                   | 10 |
| 6. Các chức năng của hệ thống .....                                                  | 10 |
| 6.1. Xử lý yêu cầu Đọc (Read Operation) .....                                        | 10 |
| 6.2. Xử lý yêu cầu Ghi (Write Operation) .....                                       | 10 |
| 6.3. Quản lý Ánh xạ và So khớp (Tag Comparison & Mapping ) .....                     | 10 |
| 6.4. Điều khiển Bộ nhớ ngoài (SDRAM Interface Control) .....                         | 10 |
| Chương III. THIẾT KẾ CHI TIẾT PHẦN CỨNG .....                                        | 11 |
| 1. Sơ đồ khối tổng thể hệ thống (Top-Level Block Diagram) .....                      | 11 |
| 2. Thiết kế khối Đường dẫn dữ liệu (Datapath Design) .....                           | 11 |
| 2.1. Tổng quan và Nguyên lý phân bố địa chỉ .....                                    | 11 |
| 2.2. Kiến trúc Datapath .....                                                        | 12 |
| 2.3. Khối xử lý địa chỉ (Address Logic) .....                                        | 12 |
| 2.4. Khối bộ nhớ đệm nội bộ (Cache Memory: Tag RAM, Data RAM, Status Bits).<br>..... | 13 |

|                                                                                       |    |
|---------------------------------------------------------------------------------------|----|
| Báo cáo Đồ án Thiết kế hệ thống số với HDL                                            |    |
| 2.5. Khối so sánh Tag (Comparator) .....                                              | 15 |
| 2.6. Khối tổng hợp lộ trình dữ liệu (Top Datapath) .....                              | 16 |
| 3. Thiết kế khối Điều khiển (Controller Design).....                                  | 18 |
| 3.1. Tổng quan và Nguyên lý điều khiển .....                                          | 18 |
| 3.2. Kiến trúc Controller .....                                                       | 19 |
| 3.3. Finite State Machine.....                                                        | 21 |
| 4. Khối kết nối hệ thống (cache_system_top) .....                                     | 25 |
| 4.1. Tổng quan và Nguyên lý thiết kế .....                                            | 25 |
| 4.2. Đặc tả giao tiếp ngoại vi (External Interfaces).....                             | 25 |
| 4.3. Phân tích kết nối nội bộ (Internal Interconnection) .....                        | 26 |
| Chương IV. MÔ PHỎNG VÀ KIỂM TRA CHỨC NĂNG .....                                       | 29 |
| 1. Kiến trúc Testbench và Môi trường mô phỏng.....                                    | 29 |
| 2. Kịch bản kiểm thử dạng sóng (Waveform Scenarios).....                              | 29 |
| 3. Kết quả mô phỏng chi tiết: .....                                                   | 33 |
| 3.1. Phân tích chu kỳ Đọc lỗi và Đọc trúng (Read Miss & Read Hit) .....               | 33 |
| 3.2. Phân tích chu kỳ Ghi trúng (Write Hit) và cơ chế bật cờ Dirty .....              | 33 |
| 3.3. Phân tích chu kỳ Trục xuất dữ liệu (Conflict Miss & Write-Back Execution). ..... | 34 |
| Chương V. TRIỂN KHAI VÀ KIỂM CHỨNG TRÊN KIT FPGA DE2 .....                            | 35 |
| 1. Giới thiệu Kit phát triển Altera DE2 và chip SDRAM vật lý .....                    | 35 |
| 2. Giải pháp Hardware Wrapper (DE2_Cache_Tester) .....                                | 35 |
| 2.1. Tích hợp IP Core In-System Sources and Probes (ISSP) giả lập CPU .....           | 36 |
| 2.2. Khối điều khiển SDRAM trung gian (sdram_controller_ip) .....                     | 37 |
| 3. Quy trình gán chân (Pin Assignment) .....                                          | 39 |
| 4. Kết quả thực nghiệm và Giao diện gỡ lỗi ISSP.....                                  | 40 |
| 4.1. Thực nghiệm chu kỳ Đọc lỗi (Read Miss) và Đọc trúng (Read Hit) .....             | 41 |
| 4.2. Thực nghiệm chính sách ghi Write-Back (Trục xuất khỏi dữ liệu bẩn).....          | 41 |
| Chương VI. TỔNG HỢP VÀ ĐÁNH GIÁ (SYNTHESIS & REPORT).....                             | 42 |
| 1. Sơ đồ mạch sau tổng hợp (RTL Viewerer Analysis) .....                              | 42 |
| 1.1. Phân tích RTL mức Top-Level (DE2_Cache_Tester).....                              | 42 |

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| Báo cáo Đồ án Thiết kế hệ thống số với HDL                                |    |
| 1.2. Phân tích RTL khối lõi hệ thống (cache_system_top).....              | 43 |
| 2. Thống kê tài nguyên chip chiếm dụng (Resource Utilization Report)..... | 43 |
| KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....                                         | 45 |
| 1. Kết luận.....                                                          | 45 |
| 2. Hướng phát triển.....                                                  | 45 |
| NGUỒN THAM KHẢO .....                                                     | 47 |

## DANH SÁCH HÌNH, BẢNG

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| Hình 1. Sơ đồ kiến trúc sử dụng Cache. ....                                 | 7  |
| Hình 2. Address_logic.v .....                                               | 13 |
| Hình 3. RTL viewer address_logic module .....                               | 13 |
| Hình 4. Cache_memory.v module .....                                         | 14 |
| Hình 5. RTL viewer cache_memory module .....                                | 15 |
| Hình 6. Comparator.v .....                                                  | 16 |
| Hình 7. RTL viewer comparator module .....                                  | 16 |
| Hình 8. Datapath_top.v .....                                                | 18 |
| Hình 9. RTL viewer datapath_top module .....                                | 18 |
| Hình 10. Sơ đồ kiến trúc Controller .....                                   | 19 |
| Hình 11. RTL viewer FSM .....                                               | 21 |
| Hình 12. Lưu đồ chuyển trạng thái .....                                     | 22 |
| Hình 13. RTL viewer controller.v .....                                      | 25 |
| Hình 14. CPU input cho cache_system_top.v .....                             | 26 |
| Hình 15. SDRAM input cho cache_system_top.v.....                            | 26 |
| Hình 16. Output DE2 Kit cho cache_system_top.v .....                        | 26 |
| Hình 17. Các tín hiệu kết nối nội bộ cho cache_system_top.v .....           | 27 |
| Hình 18. Khởi tạo datapath cho cache_system_top.v .....                     | 27 |
| Hình 19. Khởi tạo controller cho cache_system_top.v .....                   | 28 |
| Hình 20. Testbench mô phỏng kiểm tra chức năng .....                        | 33 |
| Hình 21. Kết quả mô phỏng dạng sóng chu kỳ Read .....                       | 33 |
| Hình 22. Kết quả dạng sóng mô phỏng chu kỳ Write Hit .....                  | 34 |
| Hình 23. Kết quả dạng sóng mô phỏng chu kỳ Conflict Miss & Write-Back ..... | 35 |
| Hình 24. ISSP.v do tool Quartus II khởi tạo .....                           | 37 |
| Hình 25. Sdram_controller_ip.v .....                                        | 40 |
| Hình 26. RTL viewer DE2_Cache_Tester.v .....                                | 45 |
| Hình 27. RTL viewer Cache_system_top.v .....                                | 45 |

## TÓM TẮT

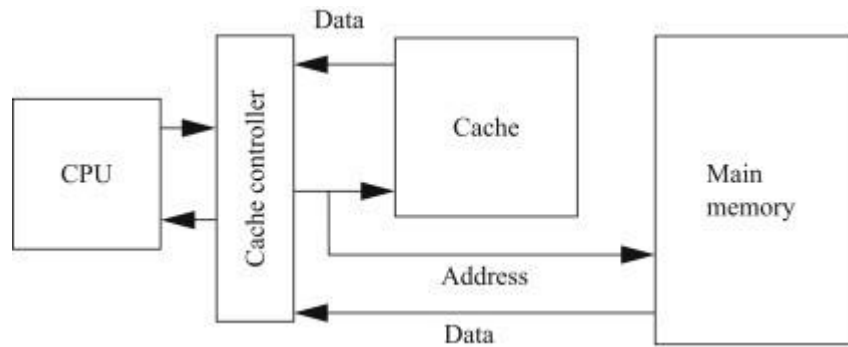
Trong các hệ thống máy tính hiện đại, sự chênh lệch ngày càng lớn về tốc độ xử lý giữa CPU và bộ nhớ chính tạo nên hiện tượng “vách ngăn bộ nhớ” (memory wall), làm giảm hiệu năng tổng thể của toàn hệ thống. Nhằm giải quyết bài toán này, đồ án tập trung vào nghiên cứu, thiết kế và hiện thực hóa một Bộ điều khiển Bộ nhớ đệm một cấp (L1 Cache Controller) cấu trúc dữ liệu phẳng trên nền tảng công nghệ FPGA. Hệ thống được xây dựng dựa trên phương pháp ánh xạ trực tiếp (Direct-Mapped) với không gian định địa chỉ 22-bit, quản lý dung lượng bộ nhớ đệm 1024 dòng với độ rộng từ nhớ 16bit. Nhóm thực hiện đã áp dụng đồng thời chính sách ghi ngược (Write-Back) kết hợp cờ trạng thái Dirty bit và chính sách cấp phát khi ghi lỗi (Write-Allocate) nhằm tối ưu hóa lưu lượng băng thông bus và giảm thiểu số lần truy cập không cần thiết xuống bộ nhớ trực chính.

Toàn bộ cấu trúc phần cứng của lõi IP Core bao gồm khối Lộ trình dữ liệu (Datapath) và khối Điều khiển máy trạng thái hữu hạn kiểu Moore (Moore FSM) được mô tả hoàn chỉnh bằng ngôn ngữ Verilog HDL. Quy trình kiểm chứng chức năng (Functional Verification) được thực hiện chặt chẽ trên phần mềm ModelSim thông qua các kịch bản mô phỏng chu kỳ Đọc/Ghi trúng (Hit) và chu kỳ Trượt xung đột (Conflict Miss) đòi hỏi trực xuất dữ liệu bản. Cuối cùng, hệ thống được tích hợp mạch Wrapper sử dụng lõi IP ISSP đóng vai trò CPU giả lập và kết nối thành công với chip nhớ ngoài SDR SDRAM dung lượng 8MB trên Kit phát triển Altera DE2 vật lý. Kết quả tổng hợp (RTL Synthesis) trên phần mềm Quartus II cho thấy thiết kế vận hành ổn định, cấu trúc mạch tối ưu khi tối ưu hóa thành công các khối RAM nhúng M4K chuyên dụng, chỉ chiếm dụng 6% tài nguyên Logic Elements của chip FPGA Cyclone II, chứng minh tính đúng đắn và khả năng ứng dụng thực tiễn của đề tài trong các hệ thống System-on-Chip (SoC) thu nhỏ.

## Chương I. TỔNG QUAN.

### 1. Tóm tắt đề tài.

Dự án tập trung vào việc thiết kế và hiện thực hóa một **Bộ điều khiển Bộ nhớ đệm (Cache Controller)** sử dụng ngôn ngữ mô tả phần cứng (HDL). Trong các hệ thống máy tính hiện đại, tốc độ xử lý của CPU nhanh hơn rất nhiều so với tốc độ truy xuất của bộ nhớ chính (SRAM/SDRAM), tạo ra hiện tượng “bottle neck”.



Hình 1. Sơ đồ kiến trúc sử dụng Cache.

Dự án này giải quyết vấn đề đó bằng cách xây dựng một lớp bộ nhớ đệm trung gian có tốc độ cao, giúp giảm thiểu thời gian chờ của CPU thông qua cơ chế lưu trữ các dữ liệu thường xuyên được truy cập. Sản phẩm cuối cùng là một lõi IP (IP Core) được mô phỏng và thực thi trên Kit FPGA DE2, có khả năng quản lý việc đọc/ghi dữ liệu giữa CPU giả lập và chip SDRAM ngoài.

### 2. Mục tiêu đề tài.

- Thiết kế thành công cấu trúc **Cache Direct-mapped**: Hiện thực hóa logic ánh xạ trực tiếp với độ dài địa chỉ **22-bit** và dữ liệu **16-bit**.
- Tối ưu hóa hiệu năng truy xuất: Đảm bảo hệ thống xử lý được các tình huống Hit (truy cập tức thời) và Miss ( nạp dữ liệu từ SDRAM) một cách tự động.
- Quản lý tính nhất quán dữ liệu: Triển khai chính sách **Write-back** kết hợp với **Dirty bit** để giảm số lần ghi không cần thiết xuống bộ nhớ chính, giúp tăng băng thông hệ thống.
- Kiểm chứng thực tế: Đảm bảo hệ thống vận hành đúng trên phần mềm Quartus II và thực thi chính xác trên phần cứng Kit DE2.

### 3. Phạm vi và giới hạn đề tài.

- Inscope (nằm trong đề tài):
  - Thiết kế bộ nhớ Cache một cấp (L1Cache).
  - Sử dụng kiến trúc Direct-Mapped (ánh xạ trực tiếp).
  - Write policy: dựa trên Hit/Miss, Write-back (ghi ngược khi hit) và Write-allocate ( nạp khi miss).
  - Giao tiếp với SDRAM trên kit DE2

Báo cáo Đồ án Thiết kế hệ thống số với HDL

- Điều khiển Cache bằng máy trạng thái hữu hạn (Moore FSM). - Outscope (ngoài đề tài):
- Kiến trúc phức tạp hơn như Set-associative hay Fully-associative.
- Hệ thống nhiều cấp Cache (L2, L3) hoặc quản lý đa nhân (Cache Coherency).

#### 4. Phân công công việc

| Thành viên              | Nhiệm vụ                                                                                                                                                              | Đánh giá                                                                                            |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Trịnh Nguyễn Thanh Bình | <ul style="list-style-type: none"><li>- Hiện thực và mô phỏng khối Datapath.</li><li>- Viết báo cáo, slide thuyết trình.</li></ul>                                    | <ul style="list-style-type: none"><li>- Mức độ đóng góp: 33.3%.</li><li>- Hoàn thành tốt.</li></ul> |
| Võ Hồ Thiên Đăng        | <ul style="list-style-type: none"><li>- Lý thuyết, nội dung, hiện thực và mô phỏng hệ thống, nạp xuống DE2 kit.</li><li>- Viết báo cáo, slide thuyết trình.</li></ul> | <ul style="list-style-type: none"><li>- Mức độ đóng góp: 33.3%.</li><li>- Hoàn thành tốt.</li></ul> |
| Bùi Quốc Bảo            | <ul style="list-style-type: none"><li>- Lý thuyết, nội dung, hiện thực và mô phỏng khối Controller.</li><li>- Viết báo cáo, slide thuyết trình.</li></ul>             | <ul style="list-style-type: none"><li>- Mức độ đóng góp: 33.3%.</li><li>- Hoàn thành tốt.</li></ul> |



## **Chương II. CƠ SỞ LÝ THUYẾT VÀ KIẾN TRÚC HỆ THỐNG.**

### **1. Tổng quan về Bộ nhớ đệm (Cache Memory).**

Bộ nhớ đệm là một bộ nhớ có tốc độ cao, dung lượng nhỏ, nằm giữa CPU và bộ nhớ chính (SDRAM). Nguyên lý hoạt động dựa trên tính cục bộ (Locality of Reference):

- Spatial Locality: CPU có xu hướng truy cập các ô nhớ gần nhau.
- Temporal Locality: Một ô nhớ vừa được truy cập có khả năng cao sẽ được truy cập lại trong tương lai gần.

### **2. Phương pháp Ánh xạ Ánh xạ trực tiếp (Direct-Mapped Cache).**

Đây là kỹ thuật ánh xạ đơn giản nhất, trong đó mỗi vị trí trong bộ nhớ chính chỉ có duy nhất một vị trí tương ứng trong Cache. Địa chỉ 22-bit từ CPU sẽ được chia thành 3 trường chính:

- Tag: Dùng để phân biệt các khối bộ nhớ chính khác nhau có cùng Index.
- Index: Xác định dòng (line) cụ thể trong Cache.
- Offset: Xác định vị trí từ (word) cụ thể trong một dòng Cache.

### **3. Chính sách ghi Write-Back và Cấp phát Write-Allocate.**

#### **3.1. Chính sách ghi Write-Back.**

- Write-back: Khi CPU thực hiện lệnh ghi và xảy ra Hit, dữ liệu chỉ được cập nhật tại Cache mà chưa ghi xuống SDRAM ngay lập tức. Việc này giúp giảm đáng kể lưu lượng truy cập bus.
- Dirty bit: Một cờ trạng thái đi kèm mỗi dòng Cache. Nếu dữ liệu trong Cache đã bị thay đổi (so với SDRAM), Dirty bit sẽ được set thành 1. Khi dòng Cache đó bị thay thế (replace), bộ điều khiển sẽ kiểm tra Dirty bit; nếu là 1, hệ thống mới thực hiện ghi ngược dữ liệu cũ xuống SDRAM.

#### **3.2. Cấp phát Write-Allocate.**

Được sử dụng khi xảy ra Write Miss. Thay vì ghi thẳng xuống SDRAM, bộ điều khiển sẽ nạp khối dữ liệu từ SDRAM vào Cache trước, sau đó mới thực hiện ghi đè tại Cache.

### **4. Công nghệ sử dụng.**

- Ngôn ngữ HDL (Verilog/SystemVerilog): Được sử dụng để mô tả cấu trúc phần cứng của Cache Controller và các thanh ghi lưu trữ. Đây là lựa chọn tối ưu để thiết kế mức RTL (Register Transfer Level).
- Phần mềm Quartus II: Công cụ chuyên dụng của Altera (nay là Intel) để tổng hợp mạch (synthesis), phân tích thời gian (timing analysis) và nạp chương trình.

Báo cáo Đồ án Thiết kế hệ thống số với HDL

- Kit DE2 (FPGA Cyclone II): Nền tảng thực thi phần cứng. Kit này tích hợp sẵn chip SDRAM ngoài, cho phép kiểm chứng thiết kế trong môi trường thực tế thay vì chỉ mô phỏng.

## 5. Thuật toán điều khiển FSM.

Hệ thống được vận hành bởi một Máy trạng thái hữu hạn (Finite State Machine) kiểu Moore để đảm bảo tính ổn định và đồng bộ theo xung clock. Các trạng thái chính bao gồm:

- IDLE: Chờ tín hiệu yêu cầu (Read/Write) từ CPU.
- COMPARE\_TAG: Trích xuất Index và so sánh Tag để xác định Hit/Miss.
- WRITE\_BACK: Nếu Miss và dòng Cache hiện tại là "Dirty", tiến hành ghi dữ liệu cũ trả lại SDRAM.
- MEM\_READ: Đọc khối dữ liệu mới từ SDRAM nạp vào Cache.
- UPDATE\_CACHE: Cập nhật dữ liệu/Tag và phản hồi hoàn tất cho CPU.

## 6. Các chức năng của hệ thống

### 6.1. Xử lý yêu cầu Đọc (Read Operation).

- Kiểm tra tính sẵn có của dữ liệu tại địa chỉ yêu cầu dựa trên việc so sánh Tag.
- Phản hồi dữ liệu ngay lập tức cho CPU nếu xảy ra tình trạng Hit.
- Tự động kích hoạt chu kỳ nạp dữ liệu từ SDRAM nếu xảy ra tình trạng Miss.

### 6.2. Xử lý yêu cầu Ghi (Write Operation).

- Thực hiện ghi dữ liệu mới vào Data RAM của Cache theo chính sách Write-back.
- Đánh dấu trạng thái dữ liệu đã bị thay đổi thông qua việc xác lập Dirty bit.

### 6.3. Quản lý Ánh xạ và So khớp (Tag Comparison & Mapping).

- Thực hiện bóc tách địa chỉ (Tag, Index, Offset) để xác định vị trí tương ứng trong Cache.
- Logic so sánh (Comparator) thực hiện kiểm tra đồng thời Tag và Valid bit để đưa ra quyết định Hit/Miss trong thời gian thực.

### 6.4. Điều khiển Bộ nhớ ngoài (SDRAM Interface Control).

- Quản lý các tín hiệu bắt tay (Handshaking) với chip SDRAM trên Kit DE2.
- Điều khiển các bus địa chỉ và dữ liệu để thực hiện các chu kỳ đọc/ghi vật lý khi có yêu cầu từ Controller.

## Chương III. THIẾT KẾ CHI TIẾT PHẦN CỨNG.

### 1. Sơ đồ khối tổng thể hệ thống (Top-Level Block Diagram).

Kiến trúc hệ thống sẽ bao gồm hai phần chính tương tác với nhau là Controller và Datapath.

### 2. Thiết kế khối Đường dẫn dữ liệu (Datapath Design).

#### 2.1. Tổng quan và Nguyên lý phân bố địa chỉ.

Khối Datapath đảm nhiệm chức năng tiếp nhận, xử lý tín hiệu địa chỉ, lưu trữ tạm thời các khối dữ liệu từ bộ nhớ trực chính và thực hiện các phép toán logic so khớp nhằm phục vụ quá trình truy xuất dữ liệu của CPU.

Trong kiến trúc kết nối với chip SDRAM dung lượng 8MB trên Kit DE2, không gian bộ nhớ vật lý được định địa chỉ bằng cấu trúc bus 22-bit dữ liệu phẳng, tương đương với  $2^{22} = 4\text{M}$  ô nhớ (độ rộng mỗi từ nhớ đạt chuẩn 16-bit). Nhằm tối ưu hóa tài nguyên phần cứng vật lý trong khi vẫn duy trì quy mô bộ nhớ Cache ở cấu hình 1024 dòng (Lines) theo chính sách ánh xạ trực tiếp (Direct-Mapping), cấu trúc 22-bit địa chỉ từ CPU phát ra ( $\text{addr}[21:0]$ ) được phân rã toán học thành hai trường thông tin chuyên biệt:

- Trường chỉ số dòng (Index Field -  $\text{addr}[9:0]$ ): Gồm 10-bit bậc thấp, thực hiện chức năng giải mã định vị trực tiếp một trong số  $2^{10} = 1024$  vị trí lưu trữ nội bộ của bộ nhớ Cache.
- Trường nhãn địa chỉ (Tag Field -  $\text{addr}[21:10]$ ): Gồm 12-bit bậc cao, đóng vai trò là định danh duy nhất đại diện cho khối dữ liệu nguồn từ SDRAM đang chiếm dụng dòng Cache tương ứng.

Về mặt phần cứng, việc duy trì quy mô 1024 dòng trên nền không gian địa chỉ mở rộng 22-bit sẽ thiết lập một tỷ lệ chia sẻ tài nguyên cố định: mỗi dòng Cache sẽ là không gian tranh chấp chung của đúng  $2^{12} = 4096$  địa chỉ nhớ khác nhau dưới SDRAM. Sự ràng buộc toán học này được thể hiện qua công thức phân bố tổng quát:

$$\text{Total Address Bits (22)} = \text{Tag Bits (12)} + \text{Index Bits (10)}$$

Để ngăn ngừa hiện tượng tràn tài nguyên logic dạng thanh ghi phân tán (Flip-Flops) trên chip FPGA Cyclone II, toàn bộ các mảng lưu trữ dung lượng lớn ( $\text{data\_ram}$  và  $\text{tag\_ram}$ ) được chuyển dịch cấu trúc sang các khối RAM nhúng chuyên dụng M4K Memory Blocks. Đồng thời, các khối RAM này được cấu hình ở chế độ đọc không đồng bộ nhằm triệt tiêu độ trễ chu kỳ, cho phép xuất dữ liệu tức thời phục vụ khối điều khiển kiểm tra trạng thái Hit/Miss.

## **2.2. Kiến trúc Datapath.**

Datapath chứa các thành phần lưu trữ và logic tính toán, thực thi các phép toán dựa trên tín hiệu điều khiển từ FSM.

- Tag RAM: Lưu trữ các bit Tag.
- Data RAM: Lưu trữ dữ liệu thực tế.
- Status Storage: Các mảng register lưu bit Valid và Dirty cho từng Index.
- Comparator: Bộ so sánh giữa Tag từ CPU và Tag đọc ra từ Tag RAM để tạo tín hiệu hit\_or\_miss.
- Address Mux: Chọn địa chỉ gửi tới SDRAM: là địa chỉ từ CPU (khi nạp dữ liệu Memory Read) hay địa chỉ ghép từ Tag cũ (khi ghi ngược - Write Back).
- Data Mux/Tri-state Buffer: Điều khiển luồng dữ liệu giữa CPU, Cache và chân SDRAM\_DQ.

Các tín hiệu Input:

- cpu\_addr [21:0]: Địa chỉ 22-bit từ CPU (để truy cập 8MB SDRAM trên DE2).
- cpu\_data\_in [15:0]: Dữ liệu CPU muốn ghi vào hệ thống.
- sdram\_data\_out [15:0]: Dữ liệu nạp vào từ chip SDRAM khi có Cache Miss.
- Tín hiệu điều khiển từ FSM: Bao gồm cache\_we, tag\_we, status\_we, sel\_addr.

Các tín hiệu Output:

- cpu\_data\_out [15:0]: Dữ liệu trả về cho CPU (sau khi Read Hit hoặc sau khi nạp xong từ SDRAM).
- sdram\_data\_in [15:0]: Dữ liệu đẩy ra bus SDRAM (khi thực hiện Write-back).
- sdram\_addr\_ext [17:0]: Địa chỉ thực tế gửi đến chip SDRAM trên kit DE2.
- hit\_or\_miss: Kết quả so sánh Tag (đưa về Controller).
- dirty\_bit\_in: Giá trị bit Dirty hiện tại của dòng Cache được truy cập (đưa về Controller).

## **2.3. Khối xử lý địa chỉ (Address Logic).**

Khối logic địa chỉ thực hiện chức năng bóc tách các trường bit Index và Tag từ CPU, đồng thời quản lý cơ chế dồn kênh địa chỉ ngõ ra gửi tới bộ nhớ ngoài.

Nguyên lý vận hành: Ở các chu kỳ hoạt động thông thường (Read/Write Hit hoặc Miss tại dòng có thuộc tính sạch Dirty = 0), bus địa chỉ gửi ra bộ nhớ ngoài (sdram\_addr\_ext) được nối trực tiếp với địa chỉ hiện tại của CPU (cpu\_addr). Ngược lại, khi xảy ra hiện tượng xung đột dòng nhớ (Conflict Miss) tại dòng đang chứa dữ liệu chưa đồng bộ (Dirty = 1), hệ thống phải thực hiện chu kỳ ghi trả (Write-Back). Khi đó, tín hiệu điều khiển sel\_addr từ khối FSM kích hoạt mức cao, module lập tức thực hiện cơ chế ghép chuỗi nhị phân (Concatenation) giữa nhãn Tag cũ đang lưu trong Cache

Báo cáo Đồ án Thiết kế hệ thống số với HDL

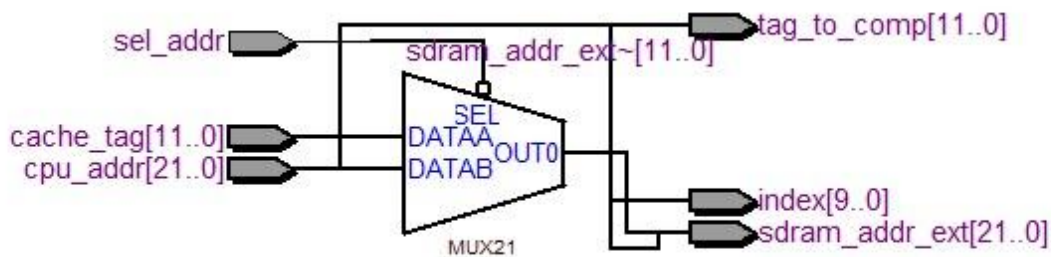
(cache\_tag) và chỉ số dòng hiện tại (index) để tái cấu trúc chính xác địa chỉ 22-bit gốc của ô nhớ dưới SDRAM:

$$sdram\_addr\_ext = \{cache\_tag, index\}$$

Cơ chế này đảm bảo dữ liệu bản được hoàn trả một cách toàn vẹn về đúng vị trí vật lý dưới bộ nhớ trực chính trước khi dòng nhớ bị ghi đè bởi khối dữ liệu mới.

```
module address_logic (  
    input [21:0] cpu_addr,  
    input [11:0] cache_tag,  
    input sel_addr,  
    output [11:0] tag_to_comp,  
    output [9:0] index,  
    output [21:0] sdram_addr_ext  
);  
    assign index = cpu_addr[9:0];  
    assign tag_to_comp = cpu_addr[21:10];  
  
    assign sdram_addr_ext = (sel_addr == 1'b0) ? cpu_addr : {cache_tag, index};  
endmodule
```

Hình 2. Address\_logic.v



Hình 3. RTL viewer address\_logic module

#### 2.4. Khối bộ nhớ đệm nội bộ (Cache Memory: Tag RAM, Data RAM, Status Bits).

Khối lưu trữ là phân vùng phần cứng chịu trách nhiệm duy trì tính toàn vẹn của dữ liệu, nhãn địa chỉ và các bit trạng thái điều khiển. Module được hiện thực hóa thông qua việc tích hợp (Instantiation) hai lõi IP RAM cấu hình phần cứng nhúng M4K:

- **data\_ram\_1024x16:** Dung lượng 1024 x 16, lưu trữ nội dung dữ liệu tạm thời.
- **tag\_ram\_1024x12:** Dung lượng 1024 x 12, lưu trữ nhãn địa chỉ phục vụ đối chiếu.

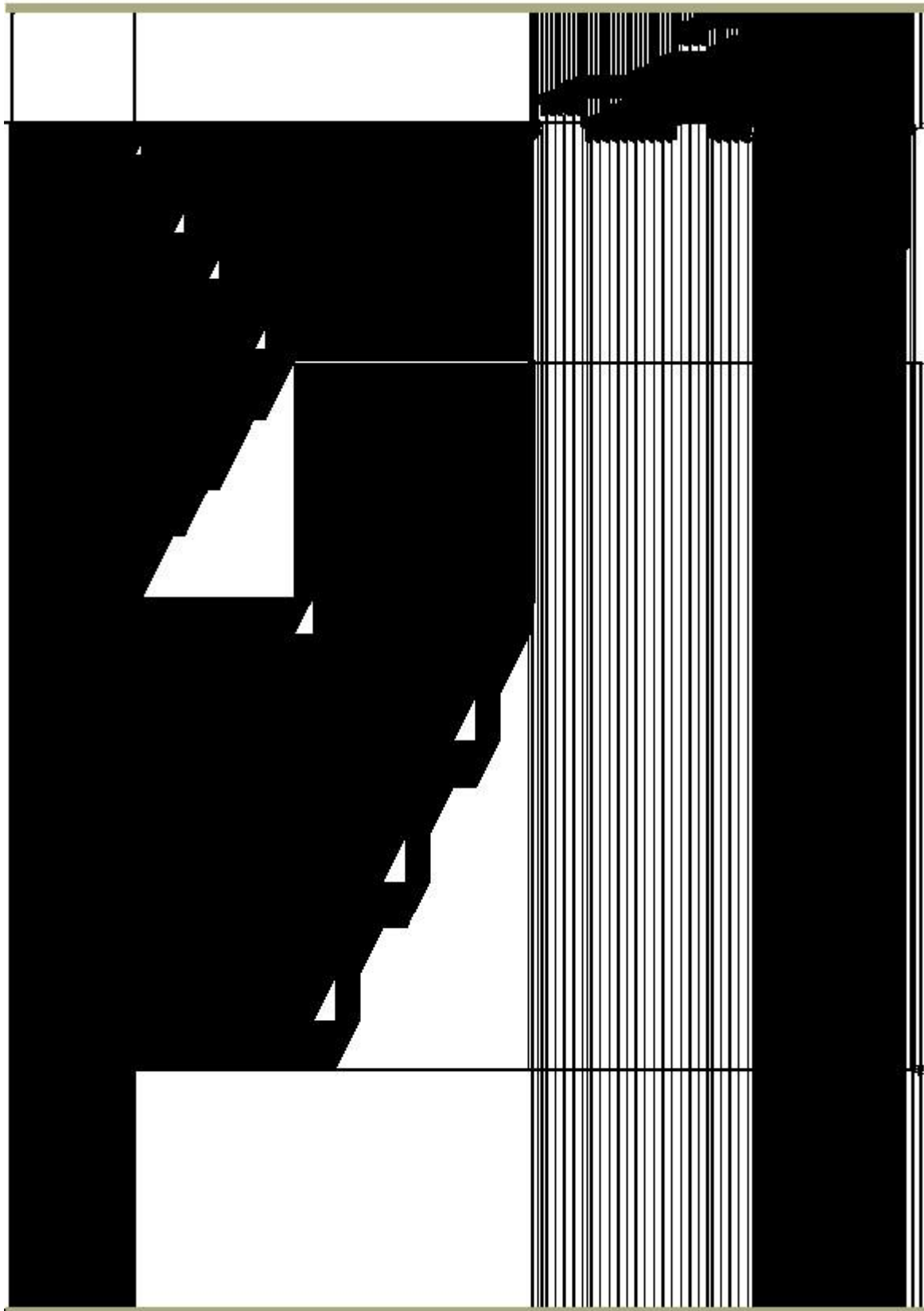
Song song đó, hai mảng thanh ghi phân tán kích thước 1024-bit (v\_bits và d\_bits) được duy trì nhằm quản lý các thuộc tính phân vùng:

## Báo cáo Đồ án Thiết kế hệ thống số với HDL

- **Bit hợp lệ (Valid Bit):** Tự động xác lập giá trị 1'b1 khi dòng nhớ được nạp dữ liệu mới từ bộ nhớ trực chính, biểu thị dữ liệu hiện tại sẵn sàng cho CPU truy xuất.
- **Bit bẩn (Dirty Bit):** Đóng vai trò giám sát tính đồng bộ giữa hai cấp bộ nhớ. Khi xảy ra chu kỳ Write Hit, dữ liệu mới cập nhật vào Cache nhưng chưa được lưu dưới SDRAM, tín hiệu điều khiển set\_dirty sẽ đưa bit thuộc tính này lên mức 1. Bit bẩn chỉ được xóa về mức 0 (clear\_dirty) khi dòng nhớ thực hiện xong chu kỳ ghi trả dữ liệu cũ và nạp mới dữ liệu sạch từ SDRAM lên.

```
1 module cache_memory (
2     input clk,
3     input reset_n,
4     input [9:0] index,
5     input [11:0] tag_in,
6     input [15:0] data_in,
7     input cache_we, tag_we, status_we,
8     input set_dirty, clear_dirty,
9     output [11:0] tag_out,
10    output [15:0] data_out,
11    output valid_out, dirty_out
12);
13    // 1. Khai báo Valid và Dirty bits bằng thanh ghi (vẫn dùng LABs vì dung lượng nhỏ)
14    reg [1023:0] v_bits;
15    reg [1023:0] d_bits;
16
17    // Xuất trạng thái hiện tại dựa trên Index
18    assign valid_out = v_bits[index];
19    assign dirty_out = d_bits[index];
20
21    // 2. Gọi module Data RAM (M4K) tạo từ MegaWizard
22    data_ram_1024x16 my_data_ram (
23        .address (index),
24        .clock (clk),
25        .data (data_in),
26        .wren (cache_we),
27        .q (data_out)
28    );
29
30    // 3. Gọi module Tag RAM (M4K) tạo từ MegaWizard
31    tag_ram_1024x12 my_tag_ram (
32        .address (index),
33        .clock (clk),
34        .data (tag_in),
35        .wren (tag_we),
36        .q (tag_out)
37    );
38
39    // 4. Logic cập nhật bit trạng thái (Valid, Dirty)
40    always @(posedge clk or negedge reset_n) begin
41        if (!reset_n) begin
42            v_bits <= {1024{1'b0}}; // ← Reset toàn bộ về 0
43            d_bits <= {1024{1'b0}}; // ← Reset toàn bộ về 0
44        end else if (status_we) begin
45            v_bits[index] <= 1'b1;
46            if (set_dirty)
47                d_bits[index] <= 1'b1;
48            else if (clear_dirty)
49                d_bits[index] <= 1'b0;
50        end
51    end
52 endmodule
```

Hình 4. Cache\_memory.v module



*Hình 5. RTL viewer cache\_memory module*

### **2.5. Khối so sánh Tag (Comparator).**

Module so khớp là đơn vị logic chịu trách nhiệm kiểm tra sự trùng khớp dữ liệu nhằm đưa ra quyết định hệ thống gặp trạng thái Trúng (Hit) hay Trượt (Miss).

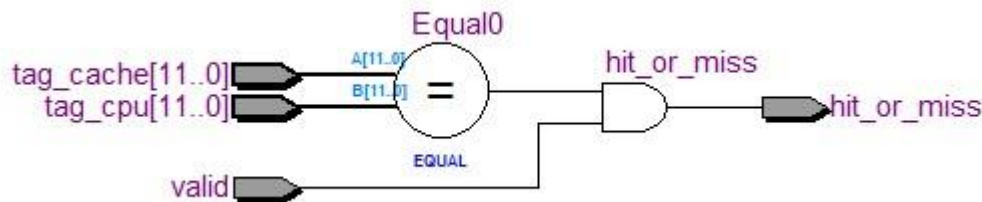
Nguyên lý vận hành: Khởi thực hiện tiếp nhận đồng thời nhãn Tag trích xuất từ yêu cầu hiện tại của CPU (tag\_cpu) và nhãn Tag đang được lưu giữ tại dòng Cache tương ứng (tag\_cache). Hàm logic xác định trạng thái Hit được cấu thành từ phép toán logic AND giữa kết quả so sánh bằng của hai nhãn Tag và trạng thái hoạt động của bit hợp lệ (valid):

$$hit\_or\_miss = (tag\_cpu == tag\_cache) \cdot valid$$

Tín hiệu ngõ ra hit\_or\_miss là tham số quyết định điều khiển chu trình của máy trạng thái. Nếu đạt mức logic 1 (Hit), CPU thực thi thao tác trực tiếp trên Cache với độ trễ bằng 0. Nếu đạt mức logic 0 (Miss), khối điều khiển sẽ tạm dừng CPU để khởi động chuỗi trạng thái tìm nạp hoặc giải phóng ô nhớ.

```
module comparator (
    input [11:0] tag_cpu,
    input [11:0] tag_cache,
    input valid,
    output hit_or_miss
);
    assign hit_or_miss = (tag_cpu == tag_cache) && (valid == 1'b1);
endmodule
```

Hình 6. Comparator.v



Hình 7. RTL viewer comparator module

## 2.6. Khối tổng hợp lộ trình dữ liệu (Top Datapath)

Khối tổng hợp đóng vai trò là lớp bọc kết nối cấp cao (Top-level Wrapper), thiết lập các đường bus liên kết và định tuyến luồng dữ liệu song hướng giữa CPU, Cache nội bộ và bộ điều khiển ngoại vi SDRAM.

Nguyên lý vận hành: Khối thực hiện cơ chế điều hướng luồng dữ liệu thông qua các bộ dồn kênh nội bộ dựa trên bản chất chu kỳ. Tại ngõ vào dữ liệu của Cache Memory, module điều khiển việc chọn nguồn ghi: nhận dữ liệu từ bus hệ thống của CPU (cpu\_data\_in) khi thực hiện cập nhật ghi đè, hoặc nhận dữ liệu từ bus bộ nhớ trực chính (sdram\_data\_out) khi đang trong trạng thái tìm nạp từ SDRAM. Đối với ngõ ra truyền xuống bộ nhớ trực chính (sdram\_data\_in), đường dữ liệu đọc từ dòng Cache hiện tại được gán cố định ra ngoài, sẵn sàng phục vụ chu kỳ ghi trả khi nhận lệnh kích hoạt từ FSM.



```

module datapath_top (
    input clk,
    input reset_n,

    // Giao tiếp với CPU
    input [21:0] cpu_addr,
    input [15:0] cpu_data_in,
    output [15:0] cpu_data_out,

    // Giao tiếp với SDRAM
    input [15:0] sdram_data_out,    // Dữ liệu nạp từ SDRAM vào
    output [15:0] sdram_data_in,    // Dữ liệu ghi ngược xuống SDRAM
    output [21:0] sdram_addr_ext,    // Địa chỉ gửi tới SDRAM (22-bit)

    // Tín hiệu điều khiển từ FSM (Controller)
    input cache_we,                // Ghi vào Data RAM
    input tag_we,                  // Ghi vào Tag RAM
    input status_we,              // Ghi vào Valid/Dirty bit
    input sel_addr,               // Chọn địa chỉ: 0 - CPU, 1 - OldTag (WriteBack)
    input set_dirty,              // Set bit Dirty khi Write Hit
    input clear_dirty,            // Xóa bit Dirty khi vừa nạp mới

    // Tín hiệu phản hồi về FSM (Controller)
    output hit_or_miss,           // Kết quả so sánh Tag
    output dirty_bit_in           // Trạng thái của dòng hiện tại
);

wire [11:0] tag_from_cache;
wire [11:0] tag_cpu;
wire [9:0] index;
wire [15:0] data_from_cache;
wire valid_bit;
wire cpu_we_select = (tag_we == 1'b0);

// 1. Module xử lý địa chỉ: Tách Tag/Index và chọn địa chỉ ra ngoài
address_logic addr_unit (
    .cpu_addr(cpu_addr),
    .cache_tag(tag_from_cache),
    .sel_addr(sel_addr),
    .tag_to_comp(tag_cpu),
    .index(index),
    .sdram_addr_ext(sdram_addr_ext)
);

```

## Báo cáo Đồ án Thiết kế hệ thống số với HDL

```
// 2. Module lưu trữ: Data RAM, Tag RAM và các bit trạng thái
cache_memory mem_unit (
    .clk(clk),
    .index(index),
    .tag_in(tag_cpu),
    .data_in(cpu_we_select ? cpu_data_in : sdram_data_out), // Chọn nguồn dữ liệu ghi vào cache
    .cache_we(cache_we),
    .tag_we(tag_we),
    .status_we(status_we),
    .set_dirty(set_dirty),
    .clear_dirty(clear_dirty),
    .tag_out(tag_from_cache),
    .data_out(data_from_cache),
    .valid_out(valid_bit),
    .dirty_out(dirty_bit_in)
);

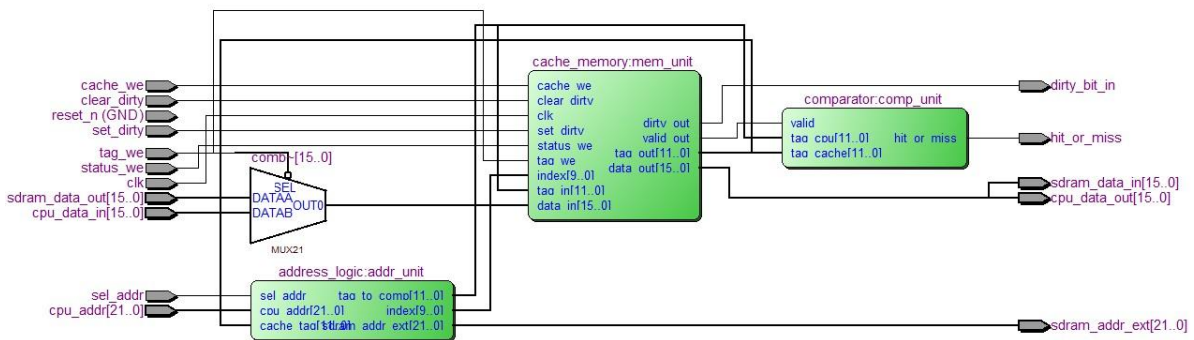
// 3. Module so sánh: Kiểm tra Hit hay Miss
comparator comp_unit (
    .tag_cpu(tag_cpu),
    .tag_cache(tag_from_cache),
    .valid(valid_bit),
    .hit_or_miss(hit_or_miss)
);

// Dữ liệu trả về CPU là dữ liệu từ Cache
assign cpu_data_out = data_from_cache;

// Dữ liệu ghi xuống SDRAM là dữ liệu cũ từ Cache (khi Write Back)
assign sdram_data_in = data_from_cache;

endmodule
```

Hình 8. Datapath\_top.v



Hình 9. RTL viewer datapath\_top module

### 3. Thiết kế khối Điều khiển (Controller Design).

#### 3.1. Tổng quan và Nguyên lý điều khiển.

Khối Controller đảm nhiệm chức năng bộ điều khiển chính của Cache, chịu trách nhiệm ra quyết định và điều phối toàn bộ luồng hoạt động giữa CPU, Cache nội bộ (Datapath) và bộ nhớ trực chính (SDRAM) thông qua việc phát sinh các tín hiệu điều khiển (Control Signals) có tính xác định theo thời gian.

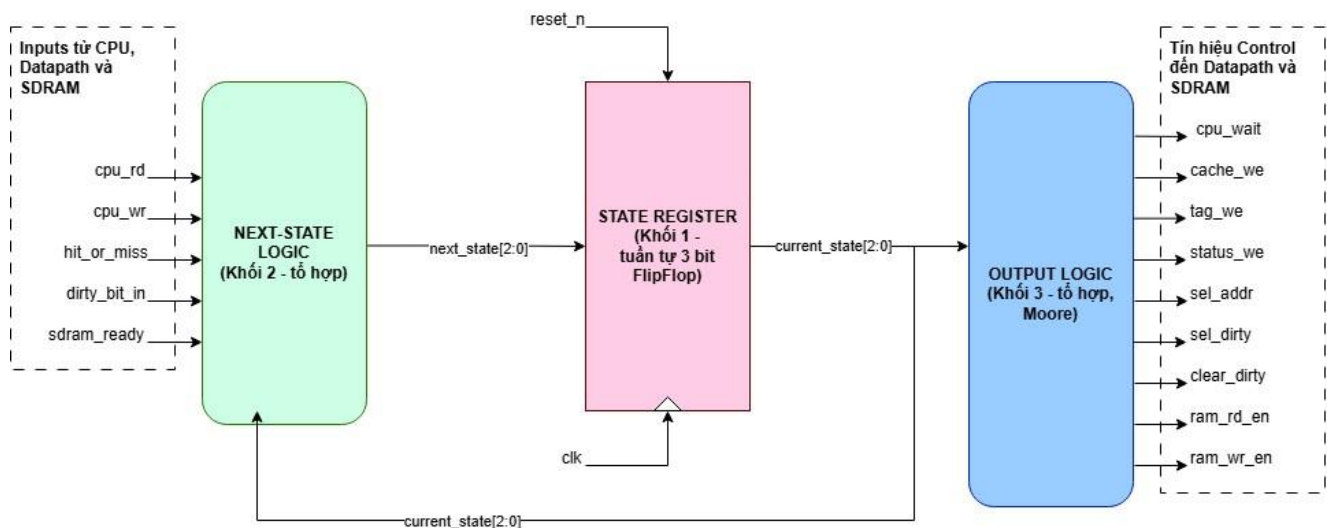
Controller được thiết kế theo mô hình máy Moore (*Moore Finite State Machine*). Đặc trưng then chốt của Moore nằm ở nguyên tắc: các tín hiệu ngõ ra chỉ phụ thuộc vào trạng thái hiện tại (*current\_state*) mà hoàn toàn không phụ thuộc trực tiếp vào giá trị tức thời của các ngõ vào. Nguyên tắc này mang lại hai ưu điểm kỹ thuật quan trọng:

## Báo cáo Đồ án Thiết kế hệ thống số với HDL

- Ổn định tín hiệu (Glitch-free Outputs): Do ngõ ra được xác định bởi thanh ghi trạng thái — một phần tử đồng bộ theo sườn xung clock — nên các tín hiệu điều khiển phát ra không bị nhiễu xung giả (glitch) do sự thay đổi bất đồng bộ của các ngõ vào gây ra.
- Có thể dự đoán được ngõ ra (Deterministic Behavior): Tại mỗi trạng thái, tổ hợp các tín hiệu điều khiển là cố định và duy nhất, giúp đơn giản hóa quá trình kiểm chứng thiết kế (verification) và gỡ lỗi (debugging).

Về mặt hiện thực phần cứng, Controller được mã hóa bằng Verilog theo khuôn mẫu thiết kế ba khối quy trình (Three-process FSM Template):

- Khối 1 - Thanh ghi trạng thái (State Register): Khối tuần tự (sequential), thực hiện lưu giữ và cập nhật giá trị trạng thái hiện tại tại mỗi sườn dương xung clock. Đồng thời xử lý logic Reset bất đồng bộ (asynchronous reset) để đưa FSM về trạng thái an toàn IDLE khi tín hiệu reset\_n được kích hoạt mức thấp.
- Khối 2 — Logic trạng thái kế tiếp (Next-State Logic): Khối tổ hợp (combinational), dựa trên trạng thái hiện tại và các tín hiệu ngõ vào phản hồi (cpu\_rd, cpu\_wr, hit\_or\_miss, dirty\_bit\_in, sdram\_ready) để xác định giá trị trạng thái kế tiếp (next\_state).
- Khối 3 — Logic ngõ ra (Output Logic): Khối tổ hợp, dựa duy nhất vào trạng thái hiện tại (tuân thủ nguyên tắc Moore) để phát sinh bộ tín hiệu điều khiển gồm 9 đường tín hiệu riêng biệt gửi tới Datapath và giao diện SDRAM bên ngoài.



Hình 10. Sơ đồ kiến trúc Controller

### 3.2. Kiến trúc Controller.

Controller chứa máy trạng thái hữu hạn và logic điều khiển, tạo ra các tín hiệu điều khiển dựa trên tín hiệu ngõ vào từ CPU và Datapath.

Các tín hiệu Input:

- clk: Xung clock hệ thống 50 MHz (từ chân CLOCK\_50 trên kit DE2), đồng bộ hóa toàn bộ hoạt động chuyển trạng thái của FSM.
- reset\_n: Tín hiệu reset tích cực mức thấp (Active-low), bắt đồng bộ. Khi reset\_n = 0, FSM được ép về trạng thái IDLE bất kể trạng thái hiện tại, đảm bảo hệ thống luôn khởi động từ một điểm xác định.
- cpu\_rd: Tín hiệu yêu cầu đọc dữ liệu từ phía Master (CPU giả lập). Khi tích cực (= 1), báo hiệu CPU đang yêu cầu đọc dữ liệu tại địa chỉ cpu\_addr.
- cpu\_wr: Tín hiệu yêu cầu ghi dữ liệu từ phía Master. Khi tích cực (= 1), báo hiệu CPU đang yêu cầu ghi dữ liệu cpu\_data\_in vào địa chỉ cpu\_addr.
- hit\_or\_miss: Tín hiệu phản hồi từ khối Comparator trong Datapath, báo cho FSM biết kết quả so sánh Tag hiện tại: mức 1 biểu thị Hit (trùng khớp), mức 0 biểu thị Miss (không trùng khớp).
- dirty\_bit\_in: Tín hiệu phản hồi từ khối Cache Memory trong Datapath, cho biết trạng thái bit Dirty của dòng Cache đang được truy cập: mức 1 biểu thị dòng đã bị sửa đổi (cần ghi ngược trước khi thay thế), mức 0 biểu thị dòng sạch.
- sdram\_ready: Tín hiệu bắt tay (handshaking) từ giao diện SDRAM bên ngoài, báo chu kỳ đọc/ghi vật lý đã hoàn tất. FSM sẽ chờ tín hiệu này trước khi chuyển trạng thái trong các chu kỳ WRITE\_BACK và MEM\_READ.

Các tín hiệu Output:

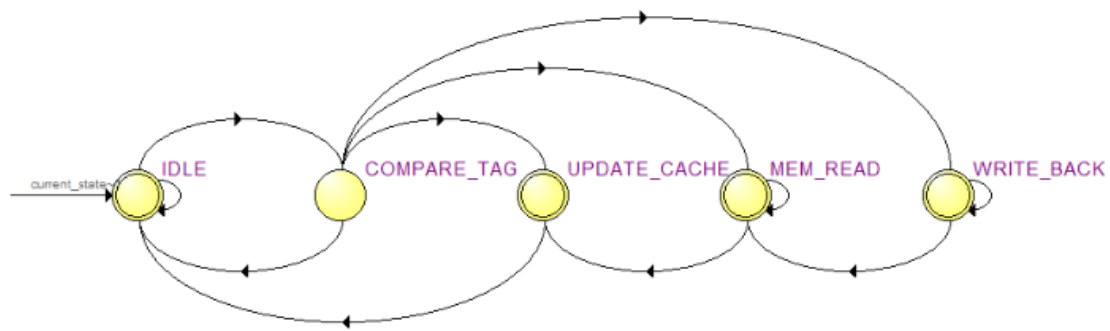
- cpu\_wait: Tín hiệu báo cho CPU tạm dừng (Stall). Khi = 1, CPU phải giữ nguyên các tín hiệu yêu cầu và chờ đợi cho đến khi Cache xử lý xong. Chỉ hạ xuống 0 tại trạng thái IDLE.
- cache\_we: Tín hiệu cho phép ghi dữ liệu vào Data RAM của Cache. Chỉ tích cực tại trạng thái UPDATE\_CACHE.
- tag\_we: Tín hiệu cho phép cập nhật giá trị Tag mới vào Tag RAM. Chỉ tích cực tại trạng thái UPDATE\_CACHE khi xảy ra Miss (cần ghi đè Tag cũ bằng Tag mới).
- status\_we: Tín hiệu cho phép cập nhật các bit trạng thái Valid và Dirty. Chỉ tích cực tại trạng thái UPDATE\_CACHE.
- sel\_addr (Mux Control): Điều khiển bộ dồn kênh địa chỉ trong Datapath. Khi = 0, chọn địa chỉ CPU gốc; khi = 1, chọn địa chỉ tái cấu trúc từ Tag cũ ghép với Index hiện tại để thực hiện Write-back.
- set\_dirty: Tín hiệu yêu cầu bật bit Dirty lên 1 cho dòng Cache hiện tại. Chỉ tích cực khi xảy ra Write Hit tại trạng thái UPDATE\_CACHE.
- clear\_dirty: Tín hiệu yêu cầu xóa bit Dirty về 0 cho dòng Cache hiện tại. Chỉ tích cực khi nạp dữ liệu mới từ SDRAM sau Miss tại trạng thái UPDATE\_CACHE.

Báo cáo Đồ án Thiết kế hệ thống số với HDL

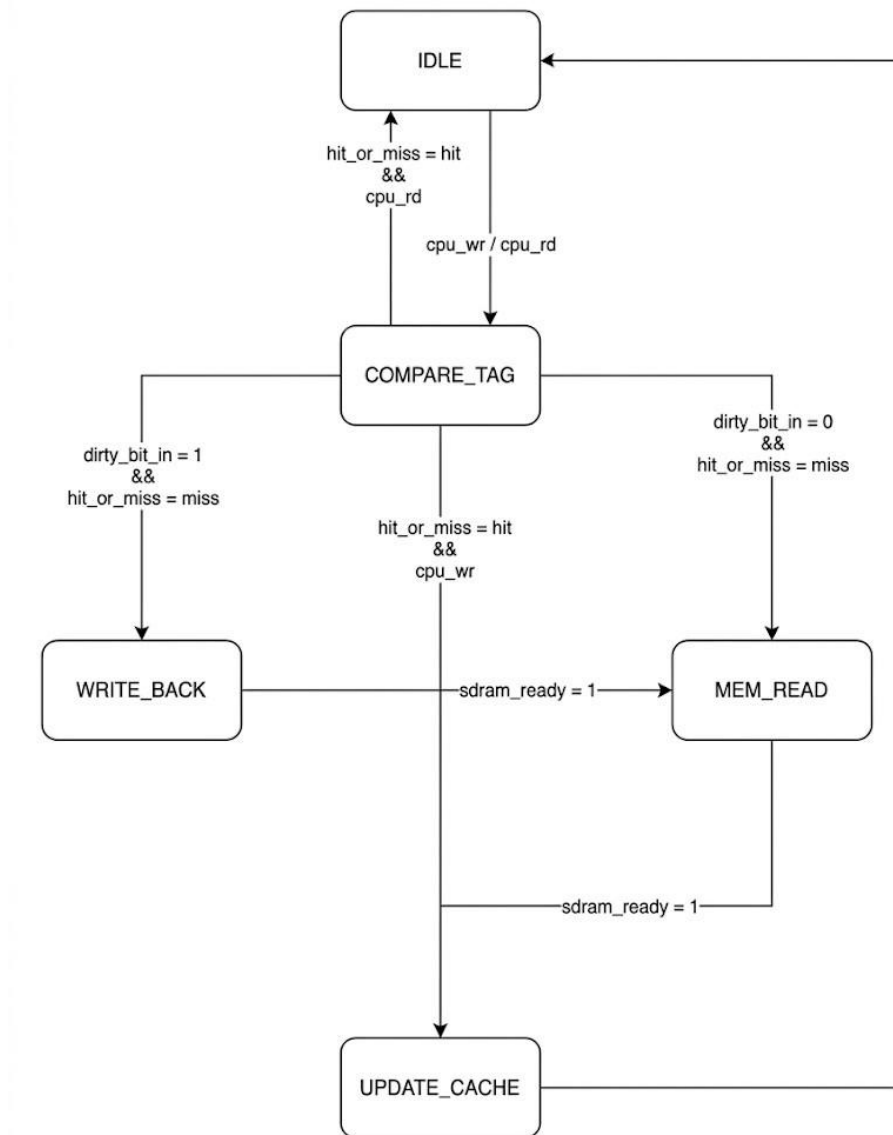
- sdram\_rd\_en: Kích hoạt chu kỳ đọc dữ liệu từ chip SDRAM trên kit. Chỉ tích cực tại trạng thái MEM\_READ.
- sdram\_wr\_en: Kích hoạt chu kỳ ghi dữ liệu xuống chip SDRAM trên kit. Chỉ tích cực tại trạng thái WRITE\_BACK.

### 3.3. *Finite State Machine.*

Sơ đồ chuyển trạng thái của hệ thống:



Hình 11. RTL viewer FSM



Hình 12. Lưu đồ chuyển trạng thái

### 3.3.1. Mã hóa trạng thái và thanh ghi trạng thái (State Encoding và State Register).

FSM của Controller sử dụng 5 trạng thái, được mã hóa bằng phương pháp mã hóa nhị phân tuần tự (Binary Encoding) với thanh ghi 3-bit:

| Trạng thái   | Mã nhị phân | Ý nghĩa                                    |
|--------------|-------------|--------------------------------------------|
| IDLE         | 3'b000      | Trạng thái nghỉ, chờ yêu cầu từ CPU        |
| COMPARE_TAG  | 3'b001      | So sánh tag để xác định Hit/Miss           |
| WRITE_BACK   | 3'b010      | Ghi dữ liệu dirty xuống SDRAM              |
| MEM_READ     | 3'b011      | Nạp khối dữ liệu mới từ SDRAM              |
| UPDATE_CACHE | 3'b100      | Cập nhật DataRAM, TagRAM và bit trạng thái |

*Việc sử dụng mã hóa nhị phân thay vì One-hot giúp tiết kiệm số lượng Flip-Flop cần thiết (chỉ cần 3 FF thay vì 5 FF), phù hợp với tài nguyên hạn chế của chip FPGA Cyclone II trên kit DE2.*

Thanh ghi trạng thái được hiện thực bằng khối always tuần tự, kích hoạt tại sườn dương xung clock (posedge clk) và hỗ trợ reset bất đồng bộ tích cực thấp (negedge reset\_n). Khi reset\_n = 0, thanh ghi lập tức trở về IDLE; còn ở chế độ bình thường, giá trị next\_state sẽ được chốt vào current\_state mỗi chu kỳ clock.

### 3.3.2. Logic chuyển trạng thái kế tiếp (Next-State Logic).

**Trạng thái IDLE (Nghỉ):** Trạng thái mặc định sau Reset. FSM chờ cho đến khi cpu\_rd = 1 hoặc cpu\_wr = 1 thì chuyển sang COMPARE\_TAG. Nếu không có yêu cầu, FSM giữ nguyên.

**Trạng thái COMPARE\_TAG (So sánh):** Đây là nút phân nhánh trung tâm của FSM, dựa trên hai tín hiệu phản hồi từ Datapath:

- *Trường hợp Hit (hit\_or\_miss = 1):* Nếu là lệnh đọc → quay về IDLE (dữ liệu đã sẵn sàng nhờ M4K RAM đọc bất đồng bộ). Nếu là lệnh ghi → chuyển sang UPDATE\_CACHE để ghi dữ liệu và bật Dirty.
- *Trường hợp Miss (hit\_or\_miss = 0):* Nếu dirty\_bit\_in = 1 → chuyển sang WRITE\_BACK để ghi trả dữ liệu cũ trước. Nếu dirty\_bit\_in = 0 → chuyển thẳng sang MEM\_READ.

**Trạng thái WRITE\_BACK (Ghi ngược):** FSM lặp tại trạng thái này cho đến khi sram\_ready = 1, sau đó chuyển sang MEM\_READ.

**Trạng thái MEM\_READ (Đọc bộ nhớ):** Tương tự, FSM chờ sram\_ready = 1 rồi chuyển sang UPDATE\_CACHE.

**Trạng thái UPDATE\_CACHE (Cập nhật):** Ghi dữ liệu vào Data RAM, cập nhật Tag (nếu Miss) và xác lập lại bit Valid/Dirty. Sau đó luôn quay về IDLE.

### 3.3.3. Logic ngõ ra theo trạng thái (Output Logic — Moore).

Theo nguyên tắc Moore, tất cả 9 tín hiệu ngõ ra chỉ phụ thuộc vào trạng thái hiện tại. Mỗi chu kỳ, toàn bộ tín hiệu được gán mặc định bằng 0 trước, sau đó chỉ những tín hiệu cần thiết mới được bật lên 1 — tránh trường hợp tín hiệu bị "treo" ngoài ý muốn.

Bảng tổng hợp giá trị ngõ ra:

| Tín hiệu | IDLE | COMPARE_TAG | WRITE_BACK | MEM_READ | UPDATE_CACHE (Hit) | UPDATE_CACHE (Miss) |
|----------|------|-------------|------------|----------|--------------------|---------------------|
| cpu_wait | 0    | 1           | 1          | 1        | 1                  | 1                   |

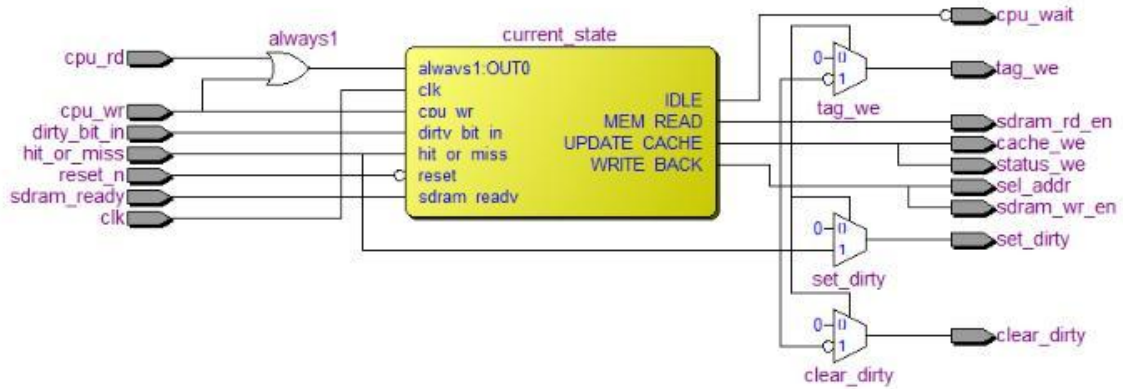
Báo cáo Đồ án Thiết kế hệ thống số với HDL

|             |   |   |   |   |   |   |
|-------------|---|---|---|---|---|---|
| cache_we    | 0 | 0 | 0 | 0 | 1 | 1 |
| tag_we      | 0 | 0 | 0 | 0 | 0 | 1 |
| status_we   | 0 | 0 | 0 | 0 | 1 | 1 |
| sel_addr    | 0 | 0 | 1 | 0 | 0 | 0 |
| set_dirty   | 0 | 0 | 0 | 0 | 1 | 0 |
| clear_dirty | 0 | 0 | 0 | 0 | 0 | 1 |
| sdram_rd_en | 0 | 0 | 0 | 1 | 0 | 0 |
| sdram_wr_en | 0 | 0 | 1 | 0 | 0 | 0 |

Phân tích chi tiết ngõ ra tại từng trạng thái:

- IDLE: Mọi tín hiệu tắt.  $\text{cpu\_wait} = 0$  cho phép CPU phát yêu cầu mới.
- COMPARE\_TAG: Chỉ bật  $\text{cpu\_wait} = 1$  để giữ CPU trong khi Datapath so sánh Tag.
- WRITE\_BACK: Bật  $\text{cpu\_wait}$ ,  $\text{sel\_addr} = 1$  (chọn địa chỉ Tag cũ ghép Index) và  $\text{sram\_wr\_en} = 1$  để ghi dữ liệu bản xuống SDRAM.
- MEM\_READ: Bật  $\text{cpu\_wait}$ ,  $\text{sel\_addr} = 0$  (dùng địa chỉ CPU) và  $\text{sram\_rd\_en} = 1$  để nạp dữ liệu mới từ SDRAM.
- UPDATE\_CACHE: Trạng thái phức tạp nhất, phân thành hai nhánh:
  - *Nhánh Write Hit ( $\text{hit\_or\_miss} = 1$ ):*  $\text{cache\_we} = 1$ ,  $\text{tag\_we} = 0$  (giữ Tag cũ),  $\text{set\_dirty} = 1$ . Khi  $\text{tag\_we} = 0$ , Datapath chọn  $\text{cpu\_data\_in}$  làm nguồn ghi.
  - *Nhánh Miss ( $\text{hit\_or\_miss} = 0$ ):*  $\text{cache\_we} = 1$ ,  $\text{tag\_we} = 1$  (ghi Tag mới),  $\text{clear\_dirty} = 1$ . Khi  $\text{tag\_we} = 1$ , Datapath chọn  $\text{sram\_data\_out}$  làm nguồn ghi.





Hình 13. RTL viewer controller.v

## 4. Khởi kết nối hệ thống (cache\_system\_top).

### 4.1. Tổng quan và Nguyên lý thiết kế.

Khởi cache\_system\_top đóng vai trò là lõi trung tâm (Core Wrapper) của toàn bộ kiến trúc bộ nhớ đệm. Tại đây, hệ thống áp dụng phương pháp thiết kế phân tầng kinh điển: Tách bạch Lộ trình dữ liệu (Data Path) và Luồng điều khiển (Control Path).

Thay vì xây dựng một module nguyên khối phức tạp xử lý đồng thời cả trạng thái chu kỳ bus lẫn tính toán cơ cấu, việc phân tách thành khối controller (FSM điều khiển trạng thái) và datapath\_top (định tuyến, so sánh và lưu trữ phần cứng) giúp tối ưu hóa khả năng kiểm tra lỗi và đóng gói cấu trúc. Đứng từ góc độ của các hệ thống ngoại vi cấp cao hơn, lõi Cache hiện ra như một thực thể đồng nhất, che giấu toàn bộ các thao tác bắt tay tín hiệu nội bộ phức tạp.

### 4.2. Đặc tả giao tiếp ngoại vi (External Interfaces).

Giao diện pins ngõ vào/ra của khối kết nối hệ thống được phân định rõ ràng thành ba nhóm chức năng chính:

- Nhóm giao tiếp với CPU: Tiếp nhận các yêu cầu truy cập xung kích đọc (cpu\_rd) hoặc ghi (cpu\_wr) đi kèm bus địa chỉ 22-bit (cpu\_addr) và bus dữ liệu ghi 16-bit (cpu\_data\_in). Đường cờ hiệu cpu\_wait đóng vai trò là tín hiệu Stall phần cứng, có nhiệm vụ ép đường ống lệnh CPU tạm dừng hoạt động trong các chu kỳ xảy ra hiện tượng Cache Miss để chờ lõi Cache thực hiện nạp dữ liệu từ bộ nhớ chính.

```

1 module cache_system_top (
2     input wire clk,
3     input wire reset_n,
4
5     // Tín hiệu giao tiếp với CPU
6     input wire cpu_rd,           // Tín hiệu yêu cầu đọc từ CPU
7     input wire cpu_wr,           // Tín hiệu yêu cầu ghi từ CPU
8     input wire [21:0] cpu_addr,  // Địa chỉ CPU yêu cầu (22-bit)
9     input wire [15:0] cpu_data_in, // Dữ liệu CPU muốn ghi
10    output wire [15:0] cpu_data_out, // Dữ liệu trả về cho CPU
11    output wire cpu_wait,         // Tín hiệu báo CPU tạm dừng (stall)

```

Hình 14. CPU input cho cache\_system\_top.v

- Nhóm giao tiếp với Main Memory (SDRAM): Đóng vai trò là một bus Master để điều phối chu kỳ truy cập với mạch điều khiển SDRAM (SDRAM Controller/IP Core). Thông qua các tín hiệu cho phép đọc (sdram\_rd\_en), cho phép ghi (sdram\_wr\_en) và bus địa chỉ mở rộng (sdram\_addr\_ext), Cache thực hiện các chu kỳ trao đổi khối dữ liệu qua lại với SDRAM ngoại vi mỗi khi cờ báo sẵn sàng (sdram\_ready) được kích hoạt.

```

13 // Tín hiệu giao tiếp với Main Memory (SDRAM Interface / IP Core)
14 input wire sdram_ready,           // Tín hiệu báo SDRAM đã sẵn sàng
15 input wire [15:0] sdram_data_out, // Dữ liệu đọc từ SDRAM lên Cache
16 output wire [15:0] sdram_data_in, // Dữ liệu từ Cache ghi ngược xuống SDRAM
17 output wire [21:0] sdram_addr_ext, // Địa chỉ cấp cho SDRAM
18 output wire sdram_rd_en,         // Tín hiệu cho phép đọc SDRAM
19 output wire sdram_wr_en,         // Tín hiệu cho phép ghi SDRAM
20

```

Hình 15. SDRAM input cho cache\_system\_top.v

- Nhóm tín hiệu quan sát phần cứng (Hardware Debugging Ports): Hai cờ trạng thái lỗi quyết định hành vi hệ thống bao gồm kết quả kiểm tra Tag (hit\_or\_miss) và trạng thái chỉnh sửa dữ liệu của dòng đệm hiện tại (dirty\_bit\_in) được trích xuất trực tiếp làm cổng ra ngoại vi. Việc đưa hai tín hiệu nội bộ này ra môi trường ngoài cho phép ánh xạ thẳng lên hệ thống đèn LED vật lý trên kit FPGA, tạo điều kiện thuận lợi cho việc kiểm chứng và phân tích trạng thái động của hệ thống phần cứng theo thời gian thực.

```

21 // Tín hiệu thông báo trên DE2 Kit
22 output wire hit_or_miss,
23 output wire dirty_bit_in

```

Hình 16. Output DE2 Kit cho cache\_system\_top.v

### 4.3. Phân tích kết nối nội bộ (Internal Interconnection).

Sự tương tác tương hỗ giữa Controller và Datapath thiết lập nên một hệ thống điều khiển phản hồi đóng vòng (Closed-loop system) thông qua các đường dây vật lý trung gian (wire nội bộ):

```

26      // Khai báo các tín hiệu kết nối nội bộ
27      wire cache_we;
28      wire tag_we;
29      wire status_we;
30      wire sel_addr;
31      wire set_dirty;
32      wire clear_dirty;

```

Hình 17. Các tín hiệu kết nối nội bộ cho cache\_system\_top.v

- Luồng phản hồi (Feedback Loop): Khối Datapath liên tục thực hiện giải mã địa chỉ CPU yêu cầu, tra cứu song song Tag RAM và đưa ra kết quả so sánh logic. Trạng thái trùng khớp Tag (hit\_or\_miss) cùng điều kiện bẩn/sạch của dòng hiện tại (dirty\_bit\_in) được đẩy ngược về các cổng ngõ vào của khối FSM Controller. Tập hợp dữ liệu này là điều kiện tiên quyết để FSM chuyển đổi trạng thái chính xác giữa chu kỳ đọc trúng (Read Hit), nạp mới bộ nhớ chính (MEM\_READ) hay thực thi trực xuất dòng bẩn (WRITE\_BACK).

```

62      // Khởi tạo Module Đường dẫn dữ liệu (Datapath)
63      datapath_top cache_dp_inst (
64          .clk(clk),
65          .reset_n(reset_n),
66
67          // Giao tiếp CPU
68          .cpu_addr(cpu_addr),
69          .cpu_data_in(cpu_data_in),
70          .cpu_data_out(cpu_data_out),
71
72          // Giao tiếp SDRAM
73          .sdram_data_out(sdram_data_out), // Luồng dữ liệu nạp từ SDRAM vào Cache Data RAM
74          .sdram_data_in(sdram_data_in),   // Luồng dữ liệu trực xuất từ Cache xuống SDRAM
75          .sdram_addr_ext(sdram_addr_ext), // Luồng địa chỉ định tuyến ra bộ nhớ chính
76
77          // Tín hiệu điều khiển từ FSM (Controller)
78          .cache_we(cache_we),
79          .tag_we(tag_we),
80          .status_we(status_we),
81          .sel_addr(sel_addr),
82          .set_dirty(set_dirty),
83          .clear_dirty(clear_dirty),
84
85          // Phản hồi về FSM (Controller)
86          .hit_or_miss(hit_or_miss),
87          .dirty_bit_in(dirty_bit_in)
88      );

```

Hình 18. Khởi tạo datapath cho cache\_system\_top.v

- Luồng điều khiển (Control Loop): Từ trạng thái hiện tại của đồ thị Moore FSM, bộ điều khiển tính toán và phát ra chùm 6 bit tín hiệu điều khiển tác vụ, bao gồm cho phép ghi dữ liệu RAM (cache\_we), cho phép ghi thẻ nhận diện (tag\_we), cho phép ghi trạng thái (status\_we), chọn nguồn địa chỉ dồn kênh (sel\_addr), cấu hình bit bẩn (set\_dirty) và xóa bit bẩn (clear\_dirty). Chùm tín hiệu này điều khiển trực tiếp các cổng MUX và chân cho phép ghi của các khối bộ nhớ thành ghi trong Datapath để thực hiện chính xác các chính sách ánh xạ trực tiếp (Direct-Mapping) và chính sách ghi lại (Write-Back).

## Báo cáo Đồ án Thiết kế hệ thống số với HDL

```
34 // Khởi tạo Module Điều Khiển (FSM Controller)
35 controller cache_ctrl_inst (
36     .clk(clk),
37     .reset_n(reset_n),
38
39     // Giao tiếp CPU
40     .cpu_rd(cpu_rd),
41     .cpu_wr(cpu_wr),
42     .cpu_wait(cpu_wait),
43
44     // Phản hồi từ Datapath
45     .hit_or_miss(hit_or_miss),
46     .dirty_bit_in(dirty_bit_in),
47
48     // Giao tiếp SDRAM
49     .sdram_ready(sdram_ready), // Ánh xạ đường sẵn sàng từ SDRAM ngoại vi
50     .sdram_rd_en(sdram_rd_en), // Lệnh đọc SDRAM phát ra từ FSM
51     .sdram_wr_en(sdram_wr_en), // Lệnh ghi SDRAM phát ra từ FSM
52
53     // Tín hiệu điều khiển Datapath
54     .cache_we(cache_we),
55     .tag_we(tag_we),
56     .status_we(status_we),
57     .sel_addr(sel_addr),
58     .set_dirty(set_dirty),
59     .clear_dirty(clear_dirty)
60 );
```

Hình 19. Khởi tạo controller cho cache\_system\_top.v

## Chương IV. MÔ PHỎNG VÀ KIỂM TRA CHỨC NĂNG.

### 1. Kiến trúc Testbench và Môi trường mô phỏng.

Để tiến hành kiểm chứng chức năng (Functional Verification) cho toàn bộ lỗi hệ thống bộ nhớ đệm trước khi hiện thực hóa trên chip FPGA, một môi trường kiểm thử toàn diện đã được xây dựng bằng ngôn ngữ Verilog HDL thông qua module `cache_system_top_tb.v`. Môi trường này sử dụng phần mềm ModelSim-Altera với độ phân giải thời gian (Timescale) thiết lập ở mức `1ns / 1ps`.

Kiến trúc của Testbench được thiết kế theo mô hình giả lập đóng vòng (Closedloop Mocking Environment) bao gồm hai khối chức năng chính bọc xung quanh khối DUT (Device Under Test):

- Khối giả lập hành vi CPU (CPU Behavioral Mocking): Sử dụng các đoạn chương trình con (task) bao gồm `cpu_read_request` và `cpu_write_request` để đóng vai trò như một bộ vi xử lý thực tế. Khối này chịu trách nhiệm cấp chu kỳ xung nhịp hệ thống (Clock Period = 10ns), phát ra các xung kích hoạt đọc/ghi (`cpu_rd`, `cpu_wr`), đẩy địa chỉ yêu cầu 22-bit (`cpu_addr`) và dữ liệu ghi 16-bit (`cpu_data_in`). Đồng thời, khối này liên tục giám sát đường cờ hiệu `cpu_wait` để thực hiện cơ chế Stall (tạm dừng CPU) đúng theo đặc tả giao thức.
- Khối giả lập phản hồi Bộ nhớ chính (SDRAM Behavioral Model): Do bộ nhớ chính SDRAM ngoài đời thực luôn có độ trễ truy cập (Memory Latency) và cần thông qua bộ điều khiển (SDRAM Controller), Testbench tích hợp một khối logic tự động phản hồi chu kỳ bus. Khi nhận được tín hiệu cho phép đọc (`sdram_rd_en`) hoặc ghi (`sdram_wr_en`) từ Cache Controller, khối này sẽ tạo ra một khoảng trễ timing mô phỏng, sau đó mới kéo cờ `sdram_ready` lên mức logic 1 và trả về hoặc ghi nhận dữ liệu, mô phỏng chính xác đáp ứng timing của hệ thống SDRAM.

### 2. Kịch bản kiểm thử dạng sóng (Waveform Scenarios).

Nhóm thiết kế xây dựng chuỗi kịch bản kiểm thử theo tiến trình logic tuyến tính để ép FSM chuyển dịch qua toàn bộ các trạng thái và kiểm tra đồng thời cả hai chính sách: Chính sách ánh xạ (Mapping Policy - Direct Mapped) và Chính sách ghi (Write Policy - Write Back / Write Allocate).

| STT | Chức năng | Input               | Trạng thái ban đầu của Cache Line | Expected Output                                                                              |
|-----|-----------|---------------------|-----------------------------------|----------------------------------------------------------------------------------------------|
| 1   | Read Miss | Đọc địa chỉ 0x00001 | Trống (Valid = 0, Dirty = 0)      | CPU bị giữ lại ( <code>cpu_wait = 1</code> ). Nạp dữ liệu từ SRAM vào Cache, cờ Valid lên 1, |

Báo cáo Đồ án Thiết kế hệ thống số với HDL

|   |                            |                                         |                                                            |                                                                                                                                                     |
|---|----------------------------|-----------------------------------------|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
|   |                            |                                         |                                                            | Dirty bằng 0.                                                                                                                                       |
| 2 | Read Hit                   | Đọc địa chỉ 0x00001                     | Đã nạp (Valid = 1, Tag khớp)                               | Không có độ trễ (cpu_wait = 0). Dữ liệu xuất thẳng từ RAM nội ra CPU ngay trong chu kỳ.                                                             |
| 3 | Write Hit                  | Ghi dữ liệu 0xAAAA vào địa chỉ 0x000001 | Đã nạp (Valid = 1, Tag khớp)                               | Dữ liệu ghi đè vào Cache nội, không ghi xuống SDRAM. Bit trạng thái thay đổi: Dirty = 1 để đánh dấu dòng dữ liệu đã bị sửa đổi so với bộ nhớ chính. |
| 4 | Conflict Miss & Write-Back | Đọc hoặc Ghi vào địa chỉ mới 0x040001   | Xung đột (Valid = 1, cùng Index nhưng khác Tag, Dirty = 1) | Phát hiện xung đột dòng bản. Kích hoạt chu kỳ trực xuất dữ liệu cũ xuống SDRAM. cpu_wait giữ mức 1 lâu nhất do tốn 2 lần truy cập SDRAM ngoại vi.   |

Testbench mô phỏng theo kịch bản:



```

1  `timescale 1ns / 1ps
2
3  module cache_system_top_tb;
4
5      // 1. Khai báo tín hiệu
6      reg clk;
7      reg reset_n;
8
9      // Giao tiếp CPU
10     reg cpu_rd;
11     reg cpu_wr;
12     reg [21:0] cpu_addr;
13     reg [15:0] cpu_data_in;
14     wire [15:0] cpu_data_out;
15     wire cpu_wait;
16
17     // Giao tiếp SDRAM
18     reg sdram_ready;
19     reg [15:0] sdram_data_out;
20     wire [15:0] sdram_data_in;
21     wire [21:0] sdram_addr_ext;
22     wire sdram_rd_en;
23     wire sdram_wr_en;
24
25     // Tín hiệu Debug ra LED vật lý
26     wire hit_or_miss;
27     wire dirty_bit_in;
28     wire [2:0] tb_current_state;
29     assign tb_current_state = DUT.cache_ctrl_inst.current_state;
30
31     // 2. Khởi tạo DUT (Device Under Test)
32     cache_system_top DUT (
33         .clk(clk),
34         .reset_n(reset_n),
35
36         .cpu_rd(cpu_rd),
37         .cpu_wr(cpu_wr),
38         .cpu_addr(cpu_addr),
39         .cpu_data_in(cpu_data_in),
40         .cpu_data_out(cpu_data_out),
41         .cpu_wait(cpu_wait),
42
43         .sdram_ready(sdram_ready),
44         .sdram_data_out(sdram_data_out),
45         .sdram_data_in(sdram_data_in),
46         .sdram_addr_ext(sdram_addr_ext),
47         .sdram_rd_en(sdram_rd_en),
48         .sdram_wr_en(sdram_wr_en),
49
50         .hit_or_miss(hit_or_miss),
51         .dirty_bit_in(dirty_bit_in)
52     );
53
54     // 3. Tạo xung Clock chu kỳ 20ns (50MHz)
55     always #10 clk = ~clk;
56

```

## Báo cáo Đồ án Thiết kế hệ thống số với HDL

```
57 // 4. Khởi giá lập phản hồi của SDRAM
58 always @(posedge clk) begin
59     if (!reset_n) begin
60         sdrd_ready <= 0;
61         sdrd_data_out <= 16'h0000;
62     end else begin
63         if (!sdrd_rd_en && !sdrd_wr_en) begin
64             sdrd_ready <= 0;
65         end
66
67         // Giả lập trễ nạp dữ liệu từ SDRAM (Trễ 2 chu kỳ clock = 40ns)
68         if (sdrd_rd_en && !sdrd_ready) begin
69             #40;
70             sdrd_ready <= 1;
71             sdrd_data_out <= {8'h00, sdrd_addr_ext[7:0], 4'h1};
72         end
73
74         // Giả lập trễ ghi ngược dữ liệu xuống SDRAM (Trễ 2 chu kỳ clock = 40ns)
75         if (sdrd_wr_en && !sdrd_ready) begin
76             #40;
77             sdrd_ready <= 1;
78             $display("[%0t] SDRAM: Da hoan tat luu dong du lieu BAN %h xuong dia chi %h", $time, sdrd_data_in, sdrd_addr_ext);
79         end
80     end
81 end
82 // 5. Các Task hỗ trợ giả lập luồng CPU
83 // Cập nhật hiển thị mã trạng thái FSM tương ứng trong lệnh in ra màn hình
84 task cpu_read_request(input [21:0] addr);
85 begin
86     @ (posedge clk);
87     cpu_rd = 1;
88     cpu_addr = addr;
89
90     @ (posedge clk);
91     cpu_rd = 0;
92
93     wait(cpu_wait == 0);
94     @ (posedge clk);
95     $display("[%0t] CPU READ: Dia chi %h -> Du lieu: %h | State hien tai: %0d", $time, addr, cpu_data_out, tb_current_state);
96 end
97 endtask
98
99 task cpu_write_request(input [21:0] addr, input [15:0] data);
100 begin
101     @ (posedge clk);
102     cpu_wr = 1;
103     cpu_addr = addr;
104     cpu_data_in = data;
105
106     @ (posedge clk);
107     cpu_wr = 0;
108
109     wait(cpu_wait == 0);
110     @ (posedge clk);
111     $display("[%0t] CPU WRITE: Ghi thanh cong %h vao dia chi %h | State hien tai: %0d", $time, data, addr, tb_current_state);
112 end
113 endtask
114
115 // 6. Kịch bản kiểm thử đồng bộ tuyến tính
116 initial begin
117     clk = 0;
118     reset_n = 0;
119     cpu_rd = 0;
120     cpu_wr = 0;
121     cpu_addr = 0;
122     cpu_data_in = 0;
123
124     $display("=====");
125     $display("    BAT DAU KIEM THU CACHE SYSTEM");
126     $display("=====");
127
128     #30 reset_n = 1;
129     #20;
130
131     // KỊCH BẢN 1: Read Miss (Địa chỉ 0x000001)
132     $display("\n--- KICH BAN 1: READ MISS ---");
133     cpu_read_request(22'h00_0001);
134
135     // KỊCH BẢN 2: Read Hit (Đọc lại chính địa chỉ 0x000001)
136     $display("\n--- KICH BAN 2: READ HIT ---");
137     cpu_read_request(22'h00_0001);
138
139     // KỊCH BẢN 3: Write Hit (Ghi 0xAAAA vào 0x000001, tạo dòng Dirty)
140     $display("\n--- KICH BAN 3: WRITE HIT (Bat co Dirty) ---");
141     cpu_write_request(22'h00_0001, 16'hAAAA);
142
143     // KỊCH BẢN 4: Conflict Miss & Write-Back (Đọc địa chỉ 0x040001 để trực xuất dòng bẩn)
144     $display("\n--- KICH BAN 4: CONFLICT MISS & WRITE-BACK ---");
145     cpu_read_request(22'h04_0001);
146
147     #100;
148     $display("\n=====");
149     $display("    MO PHONG KET THUC!");
150     $display("=====");
151     $stop;
152 end
153
154 endmodule
```

Hình 20. Testbench mô phỏng kiểm tra chức năng



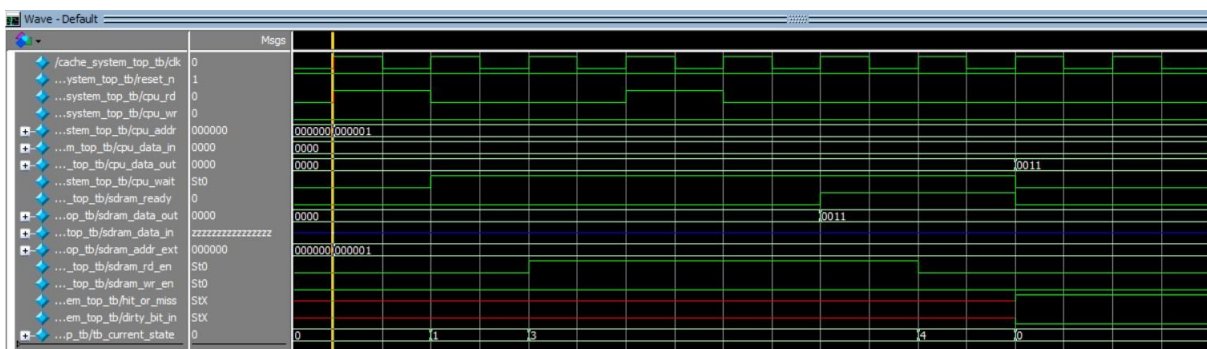
### 3. Kết quả mô phỏng chi tiết:

Sau khi chạy mô phỏng RTL trên phần mềm ModelSim, kết quả thu được tại cửa sổ Waveform và các log thông báo tại cửa sổ Transcript đã chứng minh tính đúng đắn về mặt logic của thiết kế.

#### 3.1. Phân tích chu kỳ Đọc lỗi và Đọc trúng (Read Miss & Read Hit).

Khi CPU phát lệnh đọc địa chỉ 0x000001 lần đầu tiên, hệ thống đang ở trạng thái khởi đầu, cờ valid\_bit tại dòng bộ nhớ đệm tương ứng bằng 0.

- Tại cạnh lên xung clock tiếp theo, FSM dịch chuyển từ trạng thái IDLE sang COMPARE\_TAG. Khối so sánh (Comparator) trả về tín hiệu hit\_or\_miss = 0.
- Do cờ dirty\_bit\_in = 0 (Dòng đệm sạch), FSM chuyển ngay sang trạng thái MEM\_READ. Lúc này tín hiệu sdram\_rd\_en được kéo lên mức 1 và sel\_addr = 0 để chọn địa chỉ CPU yêu cầu gửi ra ngoài.
- Sau các chu kỳ clock trì hoãn từ SDRAM mô phỏng, tín hiệu sdram\_ready tích cực. FSM dịch chuyển sang UPDATE\_CACHE. Tại đây, các cờ điều khiển ghi nội bộ cache\_we = 1, tag\_we = 1, status\_we = 1, và clear\_dirty = 1 được kích hoạt đồng thời để chụp dữ liệu từ bus SDRAM nạp vào dòng Cache, cập nhật Tag mới và xóa bit Dirty. Kết thúc chu kỳ, cpu\_wait hạ về 0, CPU nhận được dữ liệu hoàn tất.
- Ngay sau đó, khi CPU thực hiện lệnh đọc lại chính địa chỉ 0x000001, bộ so sánh trả về cờ hit\_or\_miss = 1. FSM lập tức định tuyến đường truyền đưa dữ liệu từ Cache RAM nội ra thẳng chân cpu\_data\_out và quay về IDLE chỉ trong vòng 1 chu kỳ máy mà không cần kích hoạt bus SDRAM, cờ cpu\_wait giữ nguyên ở mức 0.



Hình 21. Kết quả mô phỏng dạng sóng chu kỳ Read

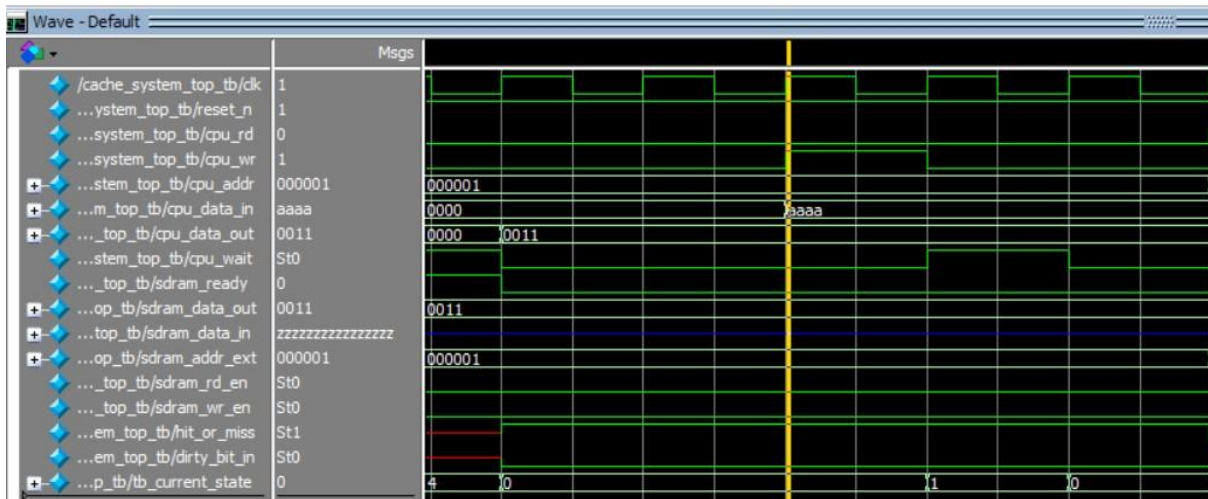
#### 3.2. Phân tích chu kỳ Ghi trúng (Write Hit) và cơ chế bật cờ Dirty.

Khi thực hiện kịch bản ghi dữ liệu 0xAAAA vào địa chỉ 0x000001, vì dòng địa chỉ này đã được nạp ở bước trước nên bộ so sánh báo Write Hit.

- FSM dịch chuyển từ COMPARE\_TAG sang UPDATE\_CACHE. Do là chu kỳ Hit, Controller tối ưu hóa lộ trình bằng cách giữ nguyên tag\_we = 0 (không

thay đổi Tag cũ) nhưng bật cờ `cache_we = 1` để ghi đè dữ liệu 0xAAAA từ CPU vào Data RAM nội bộ.

- Điểm mấu chốt của chính sách Write-Back là tín hiệu `set_dirty` được kích hoạt lên mức 1 trong chu kỳ này. Điều này điều khiển khối bit trạng thái chuyển cờ `dirty_out` của dòng Index này lên mức logic 1, đánh dấu dòng này chứa dữ liệu mới hơn bộ nhớ chính. Tín hiệu `sdrn_wr_en` vẫn giữ ở mức 0, chứng minh không có thao tác thừa xuống bộ nhớ SDRAM ngoại vi, giúp tiết kiệm băng thông bus.



Hình 22. Kết quả dạng sóng mô phỏng chu kỳ Write Hit

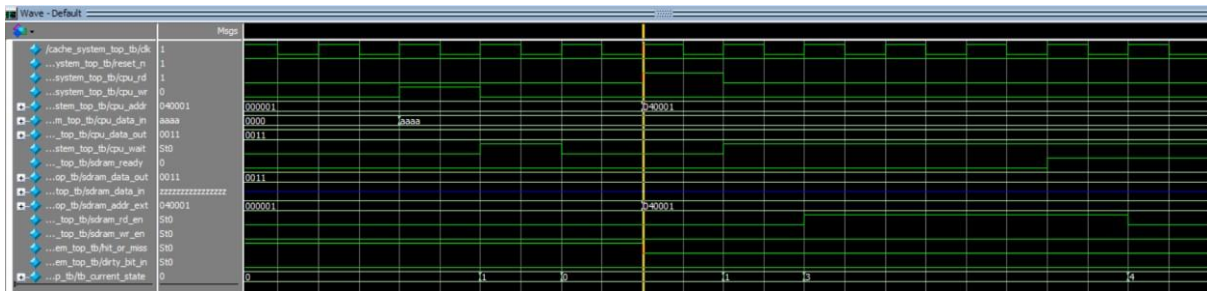
### 3.3. Phân tích chu kỳ Trục xuất dữ liệu (Conflict Miss & Write-Back Execution).

Đây là kịch bản phức tạp nhất nhằm kiểm tra giới hạn của bộ điều khiển. Khi CPU phát lệnh truy cập địa chỉ 0x040001, địa chỉ này có cùng phần Index với địa chỉ cũ nhưng có phần Tag khác biệt.

- Tại trạng thái COMPARE\_TAG, bộ so sánh nhận diện cờ `valid = 1` nhưng phần Tag không khớp, đưa ra tín hiệu lỗi `hit_or_miss = 0`. Đồng thời, do chu kỳ Write Hit trước đó đã để lại cờ `dirty_bit_in = 1`. Khối điều khiển nhận diện dòng đệm hiện tại là dòng bản và bắt buộc phải thực hiện trục xuất.
- FSM chuyển dịch sang trạng thái WRITE\_BACK. Tại đây, cờ điều khiển địa chỉ `sel_addr` được kéo lên mức 1. Lệnh này ngắt đường địa chỉ CPU và đưa địa chỉ của dòng Tag cũ ra chân `sdrn_addr_ext`, đồng thời kích hoạt `sdrn_wr_en = 1`. Dữ liệu bản 0xAAAA lưu trong dòng Cache được đẩy ngược xuống SDRAM. Cờ `cpu_wait` bị giữ chặt ở mức 1 để stall CPU.
- Sau khi SDRAM phản hồi `sdrn_ready = 1` báo cáo đã ghi nhận xong dòng bản, FSM tự động chuyển tiếp sang trạng thái MEM\_READ để hạ `sel_addr = 0`, lấy địa chỉ mới 0x040001 kích hoạt lệnh đọc `sdrn_rd_en = 1` nhằm nạp khối dữ liệu mới lên.

Báo cáo Đồ án Thiết kế hệ thống số với HDL

- Khi kết thúc giai đoạn đọc bộ nhớ chính, FSM chuyển sang UPDATE\_CACHE để ghi dữ liệu mới vào Cache, hạ bit Dirty về 0 (clear\_dirty = 1), cập nhật Tag mới (tag\_we = 1) và giải phóng cờ cpu\_wait về 0 để CPU tiếp tục hoạt động.



Hình 23. Kết quả dạng sóng mô phỏng chu kỳ Conflict Miss & Write-Back

## Chương V. TRIỂN KHAI VÀ KIỂM CHỨNG TRÊN KIT FPGA DE2.

### 1. Giới thiệu Kit phát triển Altera DE2 và chip SDRAM vật lý.

Để hiện thực hóa thiết kế hệ thống bộ nhớ đệm từ môi trường mô phỏng phần mềm xuống mạch phần cứng thực tế, nhóm nghiên cứu lựa chọn kit phát triển Altera DE2 tích hợp chip FPGA Cyclone II EP2C35F672C6. Đây là một nền tảng phần cứng học thuật mạnh mẽ, cung cấp đầy đủ các tài nguyên logic mở rộng cùng hệ thống linh kiện ngoại vi phong phú phục vụ khâu kiểm thử hệ thống nhưng.

Trong kiến trúc bo mạch của Kit DE2, bộ nhớ chính (Main Memory) được triển khai thông qua chip SDR SDRAM (Single Data Rate SDRAM) mã hiệu IS42S16400 với dung lượng 8MB. Chip nhớ ngoài này tổ chức dữ liệu theo cấu trúc 16-bit word, hoạt động đồng bộ chặt chẽ với xung nhịp hệ thống. Khác biệt cốt lõi của SDRAM so với các bộ nhớ tĩnh (SRAM) là cơ chế quản lý truy cập theo cấu trúc phân vùng Bank, Hàng (Row) và Cột (Column). Nó đòi hỏi chuỗi câu lệnh điều khiển được mã hóa thông qua tập hợp các chân chức năng đồng bộ thay vì chỉ đóng/mở cổng bằng các chân OE/WE đơn thuần. Chip SDRAM này đóng vai trò là tầng lưu trữ cấp thấp hơn, sẵn sàng cung cấp hoặc nhận lại các khối dữ liệu từ lõi Cache Controller của FPGA.

### 2. Giải pháp Hardware Wrapper (DE2\_Cache\_Tester).

Khi ánh xạ lõi thiết kế cache\_system\_top trực tiếp xuống chip phần cứng, rào cản lớn nhất nằm ở sự bất cân xứng về số lượng cổng I/O. Lõi Cache yêu cầu hệ thống giao tiếp vật lý lên tới hơn 110 chân tín hiệu khác nhau (bao gồm cả luồng giao tiếp CPU và luồng bộ nhớ ngoại vi). Trong khi đó, Kit DE2 chỉ cung cấp giới hạn các ngoại vi gát/nhấn thủ công (18 công tắc Switch và 4 nút nhấn KEY).

Để khắc phục giới hạn này và hiện thực hóa mô hình kiểm thử tự động, nhóm đã xây dựng module bao bọc phần cứng mức cao nhất mang tên DE2\_Cache\_Tester.v. Khối vật lý Wrapper này tích hợp hai giải pháp kỹ thuật chuyên sâu sau:

## 2.1. Tích hợp IP Core In-System Sources and Probes (ISSP) giả lập CPU.

Do các công tắc gạt cơ học trên board không thể đáp ứng được độ rộng bus dữ liệu và địa chỉ của CPU, nhóm đã nhúng lõi IP Core chuyên dụng In-System Sources and Probes (ISSP) của Altera. Khối IP này thiết lập một cầu nối bus dữ liệu ảo, truyền tải tín hiệu thời gian thực thông qua cáp nạp USB-Blaster kết nối trực tiếp FPGA với máy tính:

```

39 module issp (
40     probe,
41     source);
42
43     input [16:0] probe;
44     output [39:0] source;
45
46     wire [39:0] sub_wire0;
47     wire [39:0] source = sub_wire0[39:0];
48
49     altsource_probe altsource_probe_component (
50         .probe (probe),
51         .source (sub_wire0)
52         // synopsys translate_off
53         ,
54         .clrn (),
55         .ena (),
56         .ir_in (),
57         .ir_out (),
58         .jtag_state_cdr (),
59         .jtag_state_cir (),
60         .jtag_state_eldr (),
61         .jtag_state_sdr (),
62         .jtag_state_tlr (),
63         .jtag_state_udr (),
64         .jtag_state_uir (),
65         .raw_tck (),
66         .source_clk (),
67         .source_ena (),
68         .tdi (),
69         .tdo (),
70         .usrl ()
71         // synopsys translate_on
72     );
73
74     defparam
75         altsource_probe_component.enable_metastability = "NO",
76         altsource_probe_component.instance_id = "NONE",
77         altsource_probe_component.probe_width = 17,
78         altsource_probe_component.sld_auto_instance_index = "YES",
79         altsource_probe_component.sld_instance_index = 0,
80         altsource_probe_component.source_initial_value = " 0",
81         altsource_probe_component.source_width = 40;
82
83 endmodule

```

Hình 24. ISSP.v do tool Quartus II khởi tạo

- Source Port (40-bit): Đóng vai trò là tập hợp các công tắc ảo điều khiển từ phần mềm ISSP trên máy tính, được mạch Wrapper phân rã bus (Concatenation) thành: tín hiệu yêu cầu đọc `cpu_rd` (bit 39), yêu cầu ghi `cpu_wr` (bit 38), bus địa chỉ truy cập 22-bit `cpu_addr` (các bit 37..16), và bus dữ liệu ghi 16-bit `cpu_data_in` (các bit 15..0).
- Probe Port (17-bit): Đóng vai trò là tập thanh ghi quan sát, truyền ngược dữ liệu từ chân chip FPGA lên màn hình máy tính, bao gồm bus dữ liệu phản hồi `cpu_data_out` (các bit 16..1) và cờ hiệu báo bận hệ thống `cpu_wait` (bit 0).

## **2.2. Khối điều khiển SDRAM trung gian (*sdram\_controller\_ip*).**

Do giao thức điều khiển chip nhớ SDRAM đồng bộ vật lý đòi hỏi tính chính xác tuyệt đối về mặt cấu trúc thời gian (như chu kỳ tự động nạp lại Auto Refresh để duy trì điện dung ô nhớ, quản lý đa phân vùng Multi-bank, độ trễ CAS Latency), lõi Cache không thể nối dây trực tiếp ra chân chip. Nhóm thiết kế đã triển khai tích hợp module điều khiển trung gian mang tên `sdram_controller_ip`.

Khối điều khiển này nhận các yêu cầu đơn giản từ lõi Cache (như địa chỉ tuyến tính `cache_sdram_addr`, cờ cho phép đọc/ghi `sdram_rd_en/sdram_wr_en`), tự động phân rã thành địa chỉ phân vùng (`DRAM_BA`), địa chỉ hàng/cột (`DRAM_ADDR`) và phát ra các tổ hợp lệnh điều khiển chuẩn hóa vật lý gửi đến chip SDRAM (như lệnh mở dòng ACTIVE, lệnh READ, lệnh WRITE, hoặc trạng thái nghỉ NOP). Đồng thời, toàn bộ mạch đệm ba trạng thái (Tri-state Buffer) kiểm soát bus dữ liệu hai chiều `DRAM_DQ` được cô lập và đóng ngắt tự động hoàn toàn bên trong khối IP này dựa trên cờ trạng thái chu kỳ bus, giúp triệt tiêu hoàn toàn nguy cơ đụng độ tín hiệu gây ngắn mạch phần cứng.



```

1  module sdram_controller_ip (
2      // Giao tiếp với mạch Cache lõi bên trong FPGA
3      input wire clk,
4      input wire reset_n,
5      input wire [21:0] addr,      // Địa chỉ mở rộng từ Cache
6      input wire rd_en,           // Xung cho phép đọc từ Cache
7      input wire wr_en,           // Xung cho phép ghi từ Cache
8      input wire [15:0] data_in,   // Luồng dữ liệu Write-Back từ Cache
9      output reg [15:0] data_out,  // Luồng dữ liệu đọc trả về Cache
10     output reg ready,            // Tín hiệu báo sẵn sàng về Cache FSM
11
12     // Giao tiếp trực tiếp ra chân vật lý của chip SDRAM trên board DE2
13     output reg [11:0] DRAM_ADDR,
14     output reg [1:0]  DRAM_BA,
15     inout wire [15:0] DRAM_DQ,
16     output reg        DRAM_RAS_N,
17     output reg        DRAM_CAS_N,
18     output reg        DRAM_WE_N,
19     output wire       DRAM_CLK,
20     output reg        DRAM_CKE,
21     output reg        DRAM_CS_N,
22     output wire       DRAM_LDQM,
23     output wire       DRAM_UDQM
24 );
25
26 // Cấp clock và giữ mặt nạ dữ liệu luôn mở (Cho phép truy cập cả 2 byte)
27 assign DRAM_CLK = clk;
28 assign DRAM_LDQM = 1'b0;
29 assign DRAM_UDQM = 1'b0;
30
31 // Quản lý Bus hai chiều DQ bằng Tri-state buffer tích hợp nội bộ
32 reg ledr_write_mode;
33 assign DRAM_DQ = ledr_write_mode ? data_in : 16'bz;
34
35 // Bộ đếm chu kỳ để tạo độ trễ vật lý (Penalty Latency)
36 reg [2:0] cycle_cnt;
37
38 // Khai báo các trạng thái nạp/đọc của SDRAM
39 localparam STATE_IDLE = 2'b00;
40 localparam STATE_WAIT = 2'b01;
41 localparam STATE_DONE = 2'b10;
42 reg [1:0] current_state;

```

## Báo cáo Đồ án Thiết kế hệ thống số với HDL

```
44 always @(posedge clk or negedge reset_n) begin
45     if (!reset_n) begin
46         current_state <= STATE_IDLE;
47         cycle_cnt <= 0;
48         ready <= 0;
49         data_out <= 16'h0000;
50         ledr_write_mode <= 0;
51
52         // Thiết lập trạng thái nghỉ mặc định cho chip SDRAM (NOP command)
53         DRAM_CS_N <= 1'b0;
54         DRAM_CKE <= 1'b1;
55         DRAM_RAS_N <= 1'b1;
56         DRAM_CAS_N <= 1'b1;
57         DRAM_WE_N <= 1'b1;
58         DRAM_ADDR <= 12'h000;
59         DRAM_BA <= 2'b00;
60     end else begin
61         case (current_state)
62             STATE_IDLE: begin
63                 ready <= 0;
64                 if (rd_en || wr_en) begin
65                     current_state <= STATE_WAIT;
66                     cycle_cnt <= 3; // Ép độ trễ CAS Latency = 3 chu kỳ clock vật lý
67
68                     // Phân rã địa chỉ 22-bit của Cache thành cấu hình SDRAM:
69                     // 2 bits Bank Address, 12 bits Row/Column Address
70                     DRAM_BA <= addr[21:20];
71                     DRAM_ADDR <= addr[11:0];
72
73                     if (wr_en) begin
74                         ledr_write_mode <= 1; // Mở cổng đệm đẩy dữ liệu ra DQ
75                         // Lệnh GHI vật lý (WRITE Command: CS=0, RAS=1, CAS=0, WE=0)
76                         DRAM_RAS_N <= 1'b1; DRAM_CAS_N <= 1'b0; DRAM_WE_N <= 1'b0;
77                     end else begin
78                         ledr_write_mode <= 0; // Đóng cổng đệm, chuyển DQ sang nhận
79                         // Lệnh ĐỌC vật lý (READ Command: CS=0, RAS=1, CAS=0, WE=1)
80                         DRAM_RAS_N <= 1'b1; DRAM_CAS_N <= 1'b0; DRAM_WE_N <= 1'b1;
81                     end
82                 end else begin
83                     // Trạng thái không kích hoạt (Command NOP)
84                     DRAM_RAS_N <= 1'b1; DRAM_CAS_N <= 1'b1; DRAM_WE_N <= 1'b1;
85                     ledr_write_mode <= 0;
86                 end
87             end
88
89             STATE_WAIT: begin
90                 // Duy trì trạng thái NOP trong khi đợi chip nhớ ngoài đáp ứng timing
91                 DRAM_RAS_N <= 1'b1; DRAM_CAS_N <= 1'b1; DRAM_WE_N <= 1'b1;
92                 if (cycle_cnt > 1) begin
93                     cycle_cnt <= cycle_cnt - 1;
94                 end else begin
95                     current_state <= STATE_DONE;
96                     ready <= 1; // Báo cờ Sẵn sàng về cho Cache FSM
97                     if (!ledr_write_mode) begin
98                         data_out <= DRAM_DQ; // "Chụp" dữ liệu thực tế từ chip SDRAM ngoài vào
99                     end
100                 end
101             end
102
103             STATE_DONE: begin
104                 ready <= 0;
105                 ledr_write_mode <= 0;
106                 current_state <= STATE_IDLE;
107             end
108         endcase
109     end
110 end
111
112 endmodule
```

Hình 25. Sdram\_controller\_ip.v

### 3. Quy trình gán chân (Pin Assignment).

Sau khi hoàn thiện cấu trúc logic cho mạch Wrapper tổng thể, nhóm tiến hành ánh xạ các cổng tín hiệu vật lý của module DE2\_Cache\_Tester vào các chân chip thực tế trên board thông qua công cụ Pin Planner của phần mềm Quartus II bám sát theo sơ đồ mạch nguyên lý của dòng chip Cyclone II.

| Tên Port trong Mã Nguồn | Chân Chip FPGA (Cyclone II) | Loại Ngoại Vi Vật Lý Trên Board | Chức Năng Kỹ Thuật Hệ Thống                                      |
|-------------------------|-----------------------------|---------------------------------|------------------------------------------------------------------|
| CLOCK_50                | PIN_N2                      | Thạch anh Onboard 50MHz         | Cấp nguồn xung nhịp chính cho hệ thống (Chu kỳ T = 20ns)         |
| KEY[0]                  | PIN_G26                     | Push-Button (KEY0)              | Xung Reset toàn mạch (Tích cực mức thấp)                         |
| LEDG[0]                 | PIN_Y21                     | Green LED 0                     | Đèn báo hiệu trạng thái <b>Cache HIT</b> (Tìm thấy thẻ Tag khớp) |
| LEDR[0]                 | PIN_AE23                    | Red LED 0                       | Đèn báo hiệu trạng thái <b>Cache MISS</b> (Lỗi thẻ Tag)          |
| LEDR[1]                 | PIN_AF23                    | Red LED 1                       | Đèn báo hiệu trạng thái <b>Dòng đệm đang bị sửa đổi (DIRTY)</b>  |
| DRAM_CLK                | PIN_AD12                    | Chân cấp Clock cho SDRAM        | Đồng bộ tần số làm việc của chip nhớ ngoài với FPGA              |
| DRAM_RAS_N              | PIN_B14                     | Chân RAS điều khiển SDRAM       | Xung chọn địa chỉ Hàng (Row Address Strobe)                      |
| DRAM_CAS_N              | PIN_A14                     | Chân CAS điều khiển SDRAM       | Xung chọn địa chỉ Cột (Column Address Strobe)                    |
| DRAM_WE_N               | PIN_B15                     | Chân WE điều khiển SDRAM        | Tín hiệu cho phép ghi dữ liệu xuống chip nhớ ngoài               |
| DRAM_DQ[15..0]          | PIN_D10 đến PIN_AA10        | Bus dữ liệu hai chiều (inout)   | Đường luân chuyển dữ liệu đọc/ghi của bộ nhớ chính               |

#### 4. Kết quả thực nghiệm và Giao diện gỡ lỗi ISSP.

Sau khi biên dịch thành công và nạp file thực thi .sof xuống bộ cấu hình của Kit DE2 thông qua giao thức JTAG, nhóm khởi động công cụ **In-System Sources and Probes Editor** trên máy tính để thực hiện kiểm chứng phần cứng theo thời gian thực (Real-time Hardware Verification).



Tại giao diện phần mềm, nhóm tiến hành gộp nhóm (Group) các bit nhị phân thành các biến dạng Hexadecimal trực quan. Quá trình kiểm thử thực nghiệm ghi nhận đáp ứng của mạch diễn ra hoàn toàn chính xác theo các kịch bản kiểm thử:

#### **4.1. Thực nghiệm chu kỳ Đọc lỗi (Read Miss) và Đọc trúng (Read Hit).**

- Read Miss: Nhóm thiết lập địa chỉ 000001 trên ô công cụ `cpu_addr`, kích hoạt xung đọc `cpu_rd = 1` và kích tính năng *Continuous Read*. Ngay lập tức, đèn LEDR[0] (Miss) trên board bật sáng đỏ, đồng thời cờ báo bận `cpu_wait` trên giao diện phần mềm dựng lên mức cao logic 1 và giữ cứng ở đó trong một khoảng thời gian dễ dàng nhận biết bằng mắt thường. Hiện tượng này minh chứng FSM đã Stall CPU ảo thành công, bắt hệ thống tạm dừng để nhường đường cho bộ điều khiển `sdram_controller_ip` thực hiện chu kỳ timing vật lý (gồm mở hàng nhớ, độ trễ CAS Latency) để kéo dữ liệu từ chip nhớ ngoài lên. Sau khi hoàn thành nạp, `cpu_wait` rơi về 0, ô dữ liệu `cpu_data_out` hiển thị giá trị được nạp lên thành công.
- Read Hit: Nhóm nhả cờ đọc về 0, sau đó kích hoạt lại `cpu_rd = 1` tại chính địa chỉ cũ. Lần này, đèn LEDG[0] (Hit) lập tức bật sáng xanh, cờ `cpu_wait` giữ nguyên ở mức logic 0 và dữ liệu hiển thị ngay lập tức trên máy tính mà không gặp bất kỳ độ trễ nào, do dữ liệu đã nằm sẵn tại SRAM nội của FPGA.

#### **4.2. Thực nghiệm chính sách ghi Write-Back (Trục xuất khỏi dữ liệu bẩn).**

- Tạo dòng Dirty: Nhóm giữ nguyên địa chỉ 000001, nhập dữ liệu chỉnh sửa AAAA vào ô `cpu_data_in` và phát lệnh ghi `cpu_wr = 1`. Hệ thống báo chu kỳ Write Hit, dữ liệu được cập nhật vào Cache nội bộ và đèn LEDR[1] (Dirty) trên mạch vật lý bật sáng rực, chứng minh dữ liệu dòng đệm đã bị sai lệch so với bộ nhớ chính SDRAM nhưng chưa được ghi xuống.
- Xử lý Trục xuất (Eviction): Nhóm thay đổi địa chỉ sang 040001 (địa chỉ mang cùng phần Index để ánh xạ vào cùng một dòng đệm nhưng có thẻ Tag khác) và bật lệnh đọc `cpu_rd = 1`.
- Ghi nhận đáp ứng: Do dòng đệm hiện tại đang mang cờ bẩn Dirty = 1, chu kỳ trục xuất bắt đầu. Nhóm quan sát thấy cờ `cpu_wait` trên ISSP bị giữ ở mức cao logic 1 với khoảng thời gian lâu xấp xỉ gấp đôi so với chu kỳ Miss thông thường. Điều này phản ánh chính xác bản chất phần cứng: Bộ điều khiển SDRAM đang phải thực thi tuần tự hai tác vụ độc lập nối tiếp nhau: đẩy ngược khối dữ liệu bẩn AAAA lưu trong Cache xuống lưu trữ tại chip nhớ ngoài SDRAM, đóng hàng nhớ cũ, mở hàng nhớ mới, rồi mới thực hiện chu kỳ nạp dữ liệu mới của địa chỉ 040001 lên dòng đệm. Sau khi chu kỳ đóng vòng hoàn tất, đèn báo bẩn LEDR[1] tắt, dữ liệu mới hiển thị và mạch trở về trạng thái sẵn sàng.

Video demo trên DE2 Kit:

[https://drive.google.com/file/d/17rQfiixz1vAQSJZ4O\\_tnjObXVUkrfO04/view?usp=s\\_haring](https://drive.google.com/file/d/17rQfiixz1vAQSJZ4O_tnjObXVUkrfO04/view?usp=s_haring)

## Chương VI. TỔNG HỢP VÀ ĐÁNH GIÁ (SYNTHESIS & REPORT).

Sau khi hoàn tất khâu kiểm chứng chức năng trên môi trường mô phỏng ModelSim, toàn bộ hệ thống mã nguồn Verilog HDL của đồ án đã được đưa vào trình biên dịch của phần mềm **Quartus II** để thực hiện quy trình tổng hợp mạch vật lý (RTL Synthesis), tối ưu hóa cấu trúc gá đặt và định tuyến (Place & Route). Mục tiêu của chương này nhằm phân tích kiến trúc phần cứng sau tổng hợp và đánh giá hiệu năng chiếm dụng tài nguyên thực tế trên chip FPGA.

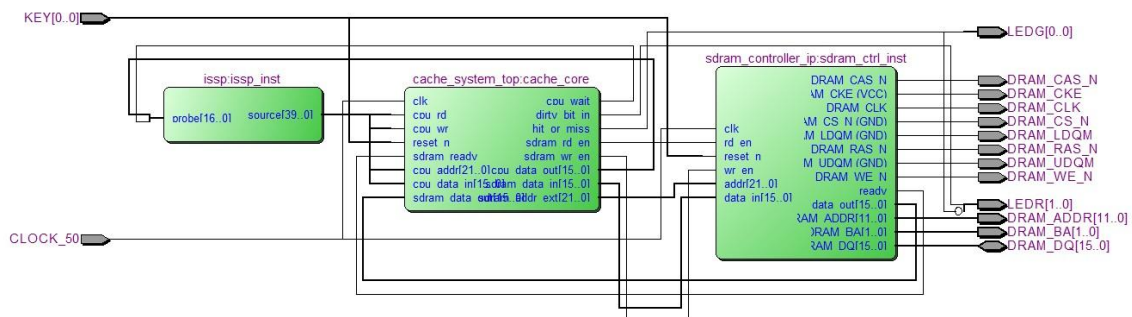
### 1. Sơ đồ mạch sau tổng hợp (RTL Viewerer Analysis).

Công cụ **RTL Viewer** trong Quartus II đóng vai trò là cầu nối trực quan giúp nhóm thiết kế kiểm tra xem trình biên dịch đã chuyển đổi các mô tả hành vi (Behavioral Description) bằng mã nguồn Verilog thành các cổng logic, bộ dồn kênh, và khối nhớ vật lý chính xác hay chưa. Sơ đồ RTL thu được phân cấp thành hai tầng mạch rõ rệt:

#### 1.1. Phân tích RTL mức Top-Level (DE2\_Cache\_Tester).

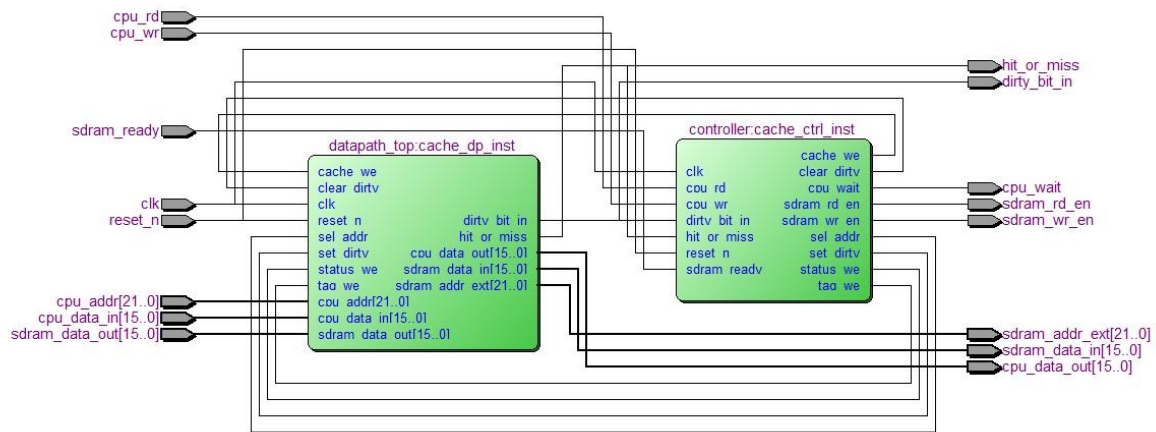
Tại sơ đồ mạch mức cao nhất, thực thể mạch được bao bọc bởi module DE2\_Cache\_Tester. Sơ đồ hiển thị rõ ràng ba khối chức năng lớn được liên kết chặt chẽ với nhau qua mạng lưới bus tín hiệu:

- Khối IP Core ISSP (issp\_inst): Nằm ở phía đầu vào của hệ thống, nhận luồng điều khiển từ máy tính qua giao tiếp JTAG và phân rã thành các đường dây rẽ nhánh cấp địa chỉ, dữ liệu cho lõi Cache.
- Khối lõi hệ thống Cache (cache\_core): Nằm ở vị trí trung tâm, tiếp nhận toàn bộ luồng giao tiếp giả lập CPU từ ISSP, thực hiện thuật toán xử lý dữ liệu đệm và xuất ra giao tiếp SDRAM Master Interface.
- Khối điều khiển SDRAM vật lý (sdram\_ctrl\_inst): Nhận lệnh Master từ Cache, chuyển đổi thành các đường điều khiển Strobe tích cực mức thấp (DRAM\_RAS\_N, DRAM\_CAS\_N, DRAM\_WE\_N) nối thẳng ra các đệm chân vật lý I/O Cell để giao tiếp với chip nhớ ngoại vi độc lập trên bo mạch.



## 1.2. Phân tích RTL khối lõi hệ thống (cache\_system\_top).

Khi thực hiện mở rộng (Push-down) vào bên trong hộp đen cache\_core, sơ đồ RTL Viewer minh chứng rõ ràng sự phân tách độc lập giữa bộ điều khiển đồ thị trạng thái FSM (cache\_ctrl\_inst) và lộ trình đường truyền dữ liệu (cache\_dp\_inst):



Hình 27. RTL viewer Cache\_system\_top.v

## 2. Thống kê tài nguyên chip chiếm dụng (Resource Utilization Report).

Sau khi trình biên dịch hoàn thành chu trình phân tích, tổng hợp và tối ưu hóa vị trí các cổng logic (Analysis & Synthesis), Quartus II xuất ra bảng thống kê chi tiết **Compilation Report**. Dưới đây là bảng số liệu thống kê tài nguyên tiêu thụ thực tế của hệ thống Cache Controller khi gán xuống dòng chip FPGA Cyclone II EP2C35F672C6 trên Kit DE2:

| Thành Phần Tài Nguyên Phần Cứng    | Số Lượng Tiêu Thụ Thực Tế | Tổng Tài Nguyên Có Sẵn Của Chip | Tỷ Lệ Chiếm Dụng (%) |
|------------------------------------|---------------------------|---------------------------------|----------------------|
| Total Logic Elements (LEs)         | 2,105                     | 33,216                          | 6%                   |
| Total Registers (Thanh ghi)        | 1235                      | 33,216                          | 4%                   |
| Total Memory Bits                  | 28,672                    | 483,840                         | 6%                   |
| Embedded Multiplier 9-bit elements | 0                         | 70                              | 0%                   |
| Total PLLs                         | 0                         | 4                               | 0%                   |
| Total Pins (Chân vật lý)           | 43                        | 475                             | 9%                   |
| Total Virtual Pins                 | 0                         | --                              | --                   |

Từ bảng số liệu thông kê biên dịch thực tế của Quartus II Flow Summary, nhóm thiết kế đưa ra các nhận xét và đánh giá chuyên môn sâu về mặt kỹ thuật vi mạch như sau:

- Tối ưu hóa tài nguyên logic (Total Logic Elements & Registers): Hệ thống tiêu tốn tài nguyên logic ở mức vừa phải với 2,105 Logic Elements (LEs), tương đương 6% năng lực xử lý tổng thể của chip FPGA Cyclone II. Trong đó, hệ thống sử dụng 1,235 thanh ghi cứng (Dedicated Logic Registers), chiếm 4% tổng số thanh ghi toàn chip. Số lượng thanh ghi tương đối lớn này được phân bổ để lưu trữ các trạng thái phức tạp của đồ thị FSM Moore, các bộ đếm chu kỳ thiết lập timing vật lý cho mạch SDRAM Controller, hệ thống thanh ghi dịch dồn kênh dữ liệu ảo ISSP và các thanh ghi đệm dòng ống lệnh (Pipeline Registers). Lượng phần tử logic tổ hợp còn lại (~870 LEs) chịu trách nhiệm thực thi các khối mạch logic giải mã điều khiển ngõ ra và mạch toán tử so sánh song song thẻ nhận diện (Tag Comparator). Tỷ lệ chiếm dụng này hoàn toàn cân đối, chứng minh mạch thiết kế đáp ứng tốt về mặt kiến trúc phần cứng của một hệ thống thu nhỏ.
- Cơ chế ánh xạ Memory Bits nội bộ: Để xây dựng các mảng bộ nhớ nội trong lõi Cache bao gồm Data RAM (lưu khối dữ liệu 16-bit word), Tag RAM (lưu thẻ nhận diện địa chỉ) và mảng lưu các bit trạng thái (Valid bit, Dirty bit), hệ thống tiêu tốn chính xác 28,672 Memory Bits, đạt tỷ lệ 6% dung lượng RAM của FPGA. Điểm mấu chốt là trình biên dịch Quartus II đã tự động nhận diện cấu trúc mảng nhớ thông qua mã nguồn Verilog và thực hiện ánh xạ (map) trực tiếp chúng vào các khối nhớ cứng chuyên dụng M4K Memory Blocks tích hợp sẵn trong lòng chip FPGA. Giải pháp này giúp triệt tiêu hoàn toàn việc phải sử dụng các flip-flop rời rạc để cấu thành bộ nhớ, giảm thiểu tối đa hiện tượng trễ định tuyến đường dây, tiết kiệm tài nguyên logic lõi và nâng cao tần số hoạt động giới hạn của hệ thống bộ nhớ đệm.
- Hiệu năng gán chân vật lý (Pin Utilization): Mạch Wrapper sử dụng tổng cộng 43 chân vật lý, đạt tỷ lệ 9% số chân I/O tối đa của dòng chip EP2C35F672C6. Tập hợp các chân này bao gồm chân cấp xung thạch anh chính CLOCK\_50 (50MHz, chu kỳ 20ns), chân nút nhấn hệ thống KEY[0], các chân gán LED vật lý xuất trạng thái ra bo mạch (LEDG[0] báo Hit, LEDR[0] báo Miss, LEDR[1] báo Dirty) và toàn bộ các chân bus địa chỉ/dữ liệu song song đồng bộ kết nối sang chip SDRAM ngoại vi vật lý trên board DE2 (DRAM\_ADDR, DRAM\_DQ, DRAM\_RAS\_N, DRAM\_CAS\_N,...). Việc tích hợp lõi IP Core CPU ảo ISSP giúp nhóm đóng gói toàn bộ giao tiếp CPU (đòi hỏi gần 40 chân) chạy ngầm qua đường JTAG của cáp nạp USBBlaster, giúp mạch tiết kiệm chân chip và không bị giới hạn bởi các công tắc gạt Switch cơ học của Kit phát triển.

## KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

### 1. Kết luận.

Sau một quá trình nghiên cứu lý thuyết, thiết kế phần cứng và triển khai thực nghiệm dưới sự hướng dẫn của giảng viên, nhóm đã hoàn thành các mục tiêu đặt ra ban đầu cho đề tài. Các kết quả cụ thể mà nhóm đã đạt được bao gồm:

- Về mặt lý thuyết và thiết kế: Nghiên cứu và làm chủ được nguyên lý tính cục bộ của dữ liệu, cơ chế bóc tách các trường địa chỉ (Tag, Index) và thuật toán điều khiển bắt tay phức tạp của chính sách ghi Write-Back / Write-Allocate. Thiết kế thành công cấu trúc phân tầng giữa Datapath và Controller theo mô hình Moore FSM đảm bảo ngõ ra ổn định, không nhiễu xung giả.
- Về mặt mô phỏng và kiểm chứng: Xây dựng môi trường testbench trên ModelSim. Các dạng sóng thu được đã chứng minh độ chính xác của mạch logic khi xử lý các kịch bản Hit/Miss, đặc biệt là cơ chế Stall CPU (`cpu_wait = 1`) để thực hiện tuần tự hai tác vụ độc lập: ghi trả khối dữ liệu cũ bị trục xuất xuống bộ nhớ chính trước khi nạp khối dữ liệu mới.
- Về mặt triển khai phần cứng: Hiện thực hóa thành công mô hình nhúng trên Kit FPGA DE2 vật lý. Nhóm đã giải quyết được rào cản giới hạn chân I/O bằng giải pháp cầu nối ảo JTAG thông qua lõi IP ISSP để giao tiếp thời gian thực với máy tính. Đồng thời, việc tích hợp khối `sdram_controller_ip` trung gian đã giúp điều khiển chip nhớ SDRAM IS42S16400 bên trên kit.
- Về mặt tối ưu hóa tài nguyên: Thiết kế đạt độ tinh gọn cao khi chỉ tiêu tốn 2,105 LEs (6%) và 28,672 Memory Bits (6%) nhờ cơ chế tự động nhận diện và ánh xạ trực tiếp các mảng bộ nhớ đệm vào khối RAM cứng M4K trên chip Cyclone II.

Bên cạnh những thành công đó, hệ thống vẫn tồn tại một số giới hạn như: kiến trúc ánh xạ trực tiếp (Direct-Mapped) dễ dẫn đến hiện tượng tranh chấp dòng đệm liên tục nếu CPU truy cập liên tiếp các địa chỉ trùng Index, hệ thống hiện tại mới chỉ kiểm thử tính đúng đắn của dữ liệu trên Kit FPGA DE2 thông qua CPU giả lập ISSP, chưa đánh giá được hiệu năng thực tế khi chạy với các chương trình phần mềm (benchmark) phức tạp, và mới chỉ thử nghiệm ở cấu hình Cache một cấp (L1 Cache) gắn với cấu trúc CPU giả lập đơn giản.

### 2. Hướng phát triển.

Để nâng cao hiệu năng và tiệm cận gần hơn với các kiến trúc vi xử lý thương mại trong thực tế, nhóm đề xuất các hướng phát triển tiếp theo cho đề tài như sau:

- Nâng cấp kiến trúc ánh xạ: Cải tiến khối Datapath và logic so khớp từ Ánh xạ trực tiếp (Direct-Mapped) lên kiến trúc Ánh xạ tập hợp liên kết (Set-Associative Cache, ví dụ: 2-way hoặc 4-way) kết hợp thuật toán thay thế dòng trống LRU

Báo cáo Đồ án Thiết kế hệ thống số với HDL

(Least Recently Used) nhằm giảm thiểu tối đa tỷ lệ lỗi trượt xung đột (Conflict Miss).

- Mở rộng quy mô hệ thống Cache đa cấp: Nghiên cứu và triển khai cấu trúc Cache nhiều tầng (bao gồm L1 Cache tốc độ cao bất đồng bộ kết hợp với L2/L3 Cache dung lượng lớn hơn) để tối ưu hóa hơn nữa phân cấp bộ nhớ.
- Tích hợp vào lõi CPU thực tế: Đóng gói lõi IP Cache Controller này để làm tầng bộ nhớ đệm chính thức cho các vi xử lý mã nguồn mở như RISC-V 32-bit CPU, thay vì sử dụng ISSP giả lập, tiến tới xây dựng một hệ thống System-on-Chip (SoC) hoàn chỉnh trên FPGA.
- Hiện thực hóa cơ chế quản lý đa nhân: Nghiên cứu các giao thức duy trì tính nhất quán dữ liệu (Cache Coherency Protocols) như MSI hoặc MESI để sẵn sàng mở rộng cho các hệ thống phần cứng đa lõi (Multi-core processors).

## NGUỒN THAM KHẢO

1. Altera Corporation, *DE2 Development and Education Board User Manual*, Core IP System Library.
2. David A. Patterson and John L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, RISC-V Edition.
3. Slide bài giảng môn học CE213: Thiết kế hệ thống số với HDL.